Bazele inteligenței artificiale

Laborator – Grigore Ionuț

Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models → **today's lab**
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

Content

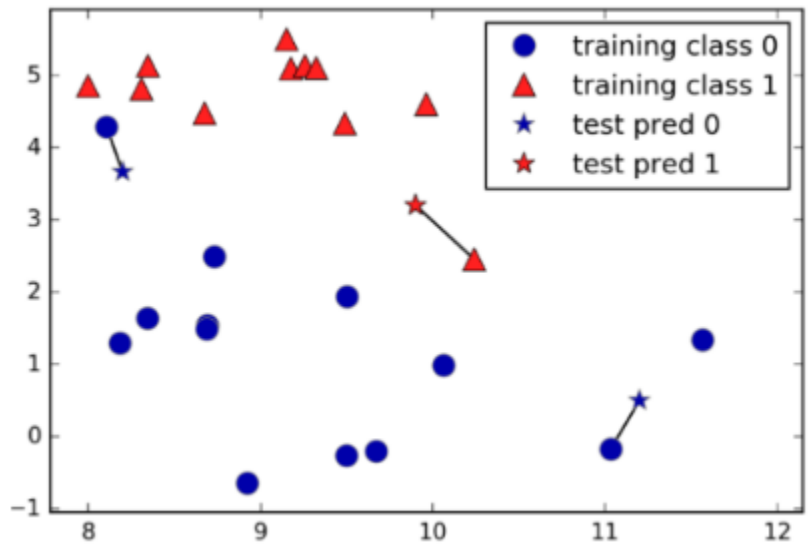
- *k*-Nearest neighbors
- Linear SVM classification
- Nonlinear SVM classification
- SVM regression

1) *k*-Nearest neighbors

Classification

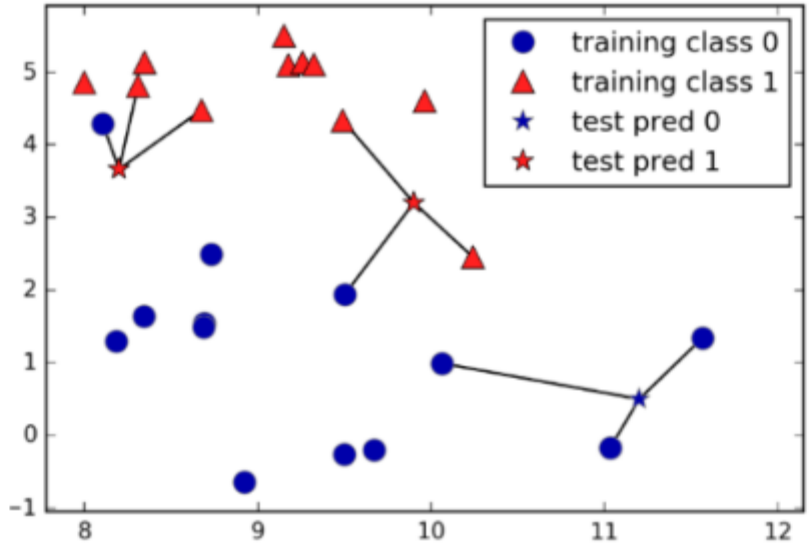
Nonparametric! We don't have parameters ***w***!

- building the model consists only of storing the training dataset;
- to make a prediction for a new data point, the algorithm finds the closest data points in the training dataset - its "nearest neighbors";



Predictions made by the one-nearest neighbor model.

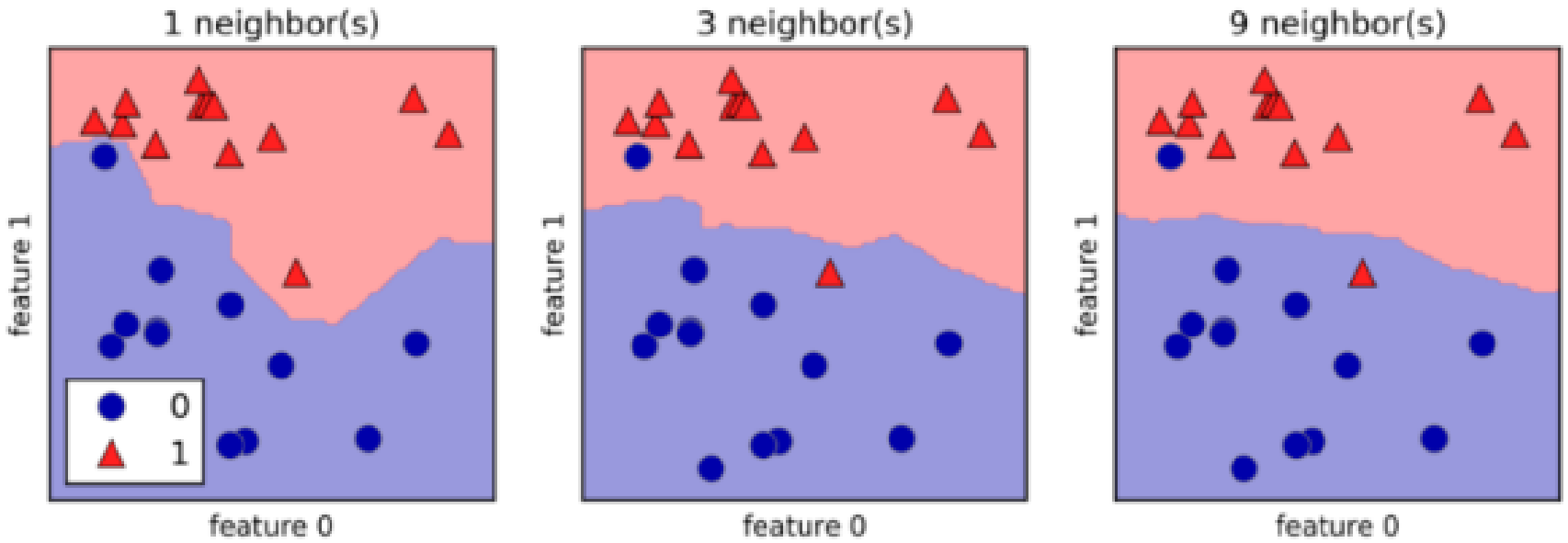
- instead of considering only the closest neighbor, we can also consider an arbitrary number, k, of neighbors;
- when considering more than one neighbor, we use *voting* to assign a label;
- we count how many neighbors belong to class 0 and how many neighbors belong to class 1;
- we then assign the class that is more frequent;



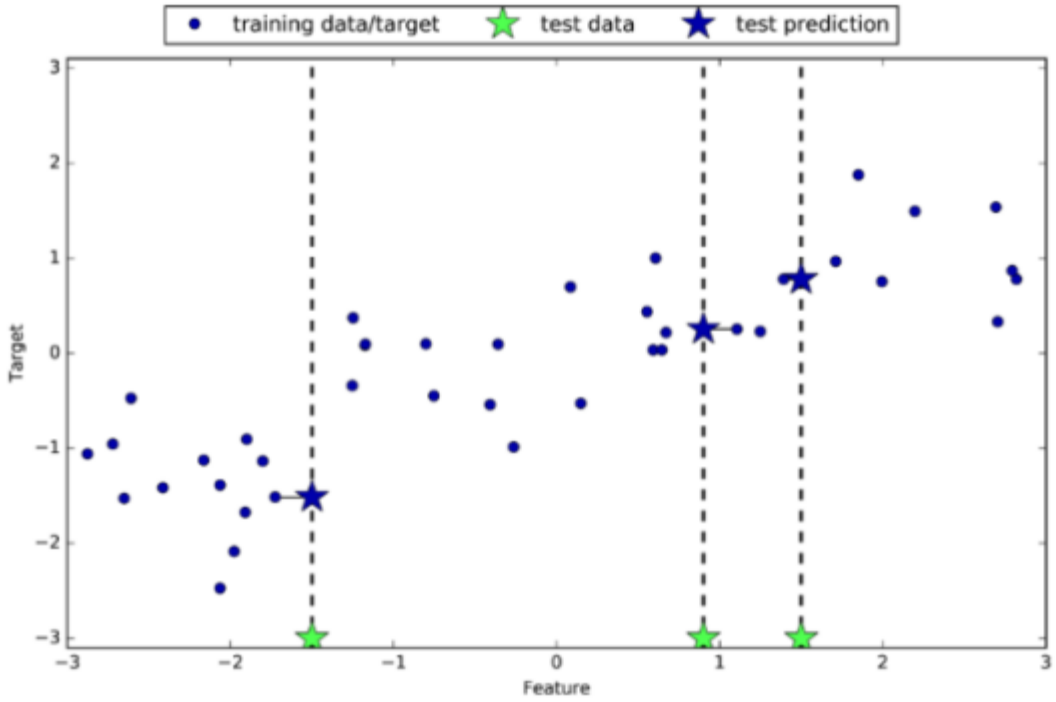
Predictions made by the three-nearest neighbors model

- we can also illustrate the prediction for all possible test points in the $xy-$plane;

- we color the plane according to the class that would be assigned to a point in this region;

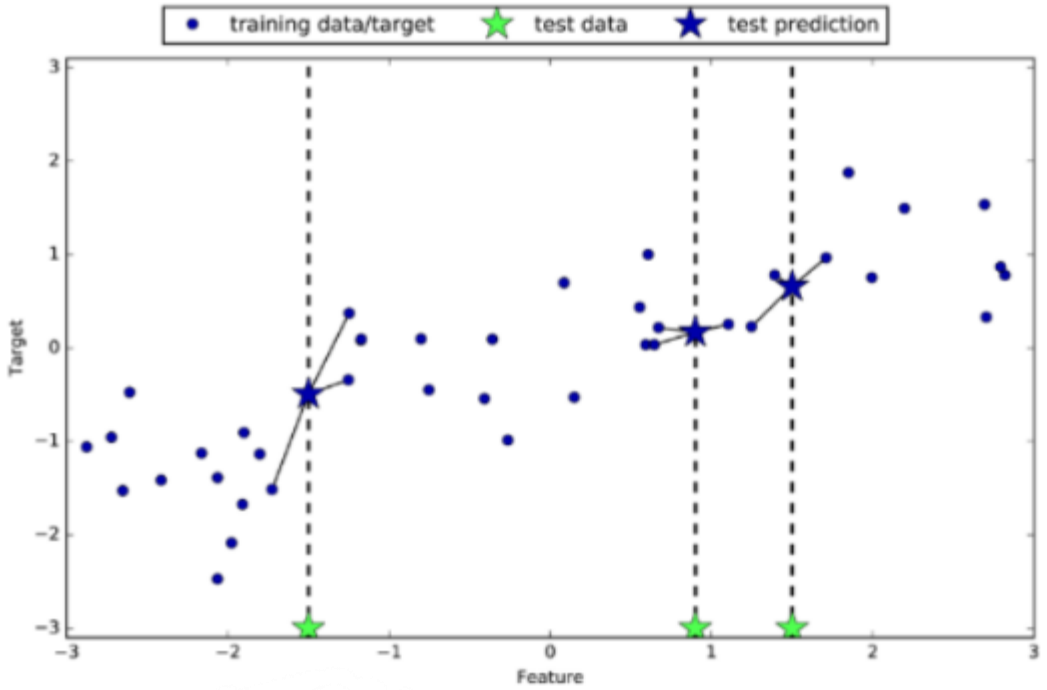


Regression



Predictions made by one-nearest neighbor regression

For each green star, it looks for the nearest blue dot, and whatever the value of that blue dot is (from OY axis), it becomes the prediction (blue star). It's like asking your closest friend for advice and following exactly what they say.



Predictions made by three-nearest neighbors regression

For each green star, it looks at the three closest blue dots, takes their values (from OY axis), and averages them to come up with a prediction (blue star). This way, you're getting a more balanced opinion because you're asking a few more people.

2 important things are essential for *k*-Nearest neighbors:

- **the number of neighbors**
- how we measure the **distance** between data points (Euclidean distance is commonly used)

Real-World Uses of k-Nearest Neighbors (k-NN)

Where k-NN is used (some examples):

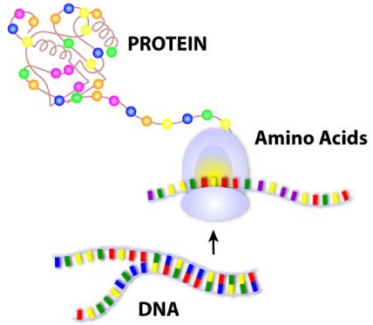
- **Healthcare:** Predicting disease based on patient similarity (e.g., diabetes, heart disease).



- **Recommendation systems:** "Users who liked this also liked…" (product or movie similarity).

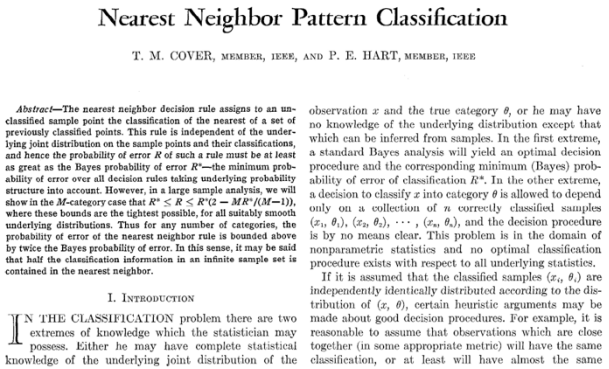


- **Bioinformatics:** Classifying genes or protein sequences by similarity.



Notable papers

- Cover & Hart (1967) — Nearest neighbor pattern classification → foundational work.



Nearest Neighbor Pattern Classification

T. M. COVER, MEMBER, IEEE, AND P. E. HART, MEMBER, IEEE

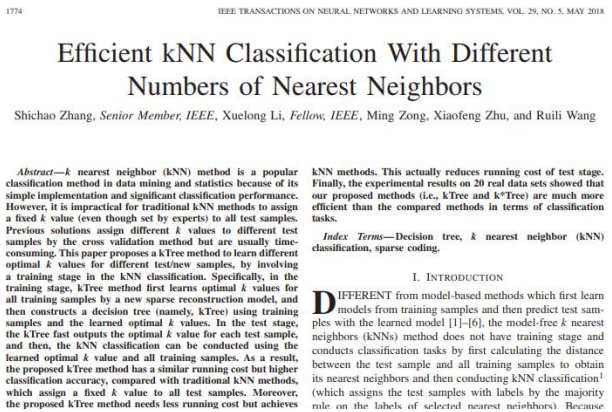
Abstract—The nearest neighbor decision rule assigns to an unclassified sample point the classification of the nearest of a set of previously classified points. This rule is independent of the underlying joint distribution on the sample points and their classifications, and hence the probability of error R of such a rule must be at least as great as the Bayes probability of error R^*—the minimum probability of error over all decision rules taking underlying probability structure into account. However, in a large sample analysis, we will show in the M-category case that $R^* \leq R \leq R^*(2 - MR^*/(M-1))$, where these bounds are the tightest possible, for all suitably smooth underlying distributions. Thus for any number of categories, the probability of error of the nearest neighbor rule is bounded above by twice the Bayes probability of error. In this sense, it may be said that half the classification information in an infinite sample set is contained in the nearest neighbor.

I. Introduction

IN THE CLASSIFICATION problem there are two extremes of knowledge which the statistician may possess. Either he may have complete statistical knowledge of the underlying joint distribution of the observation x and the true category θ, or he may have no knowledge of the underlying distribution except that which can be inferred from samples. In the first extreme, a standard Bayes analysis will yield an optimal decision procedure and the corresponding minimum (Bayes) probability of error of classification R^*. In the other extreme, a decision to classify x into category θ is allowed to depend only on a collection of n correctly classified samples $(x_1, \theta_1), (x_2, \theta_2), \cdots, (x_n, \theta_n)$, and the decision procedure is by no means clear. This problem is in the domain of nonparametric statistics and no optimal classification procedure exists with respect to all underlying statistics.

If it is assumed that the classified samples (x_i, θ_i) are independently identically distributed according to the distribution of (x, θ), certain heuristic arguments may be made about good decision procedures. For example, it is reasonable to assume that observations which are close together (in some appropriate metric) will have the same classification, or at least will have almost the same

- Zhang (2017) — Efficient kNN classification with different numbers of nearest neighbors



1774 IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, VOL. 29, NO. 5, MAY 2018

Efficient kNN Classification With Different Numbers of Nearest Neighbors

Shichao Zhang, *Senior Member, IEEE*, Xuelong Li, *Fellow, IEEE*, Ming Zong, Xiaofeng Zhu, and Ruili Wang

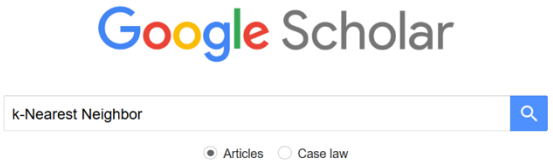
Abstract—k nearest neighbor (kNN) method is a popular classification method in data mining and statistics because of its simple implementation and significant classification performance. However, it is impractical for traditional kNN methods to assign a fixed k value (even though set by experts) to all test samples. Previous solutions assign different k values to different test samples by the cross validation method but are usually time-consuming. This paper proposes a kTree method to learn different optimal k values for different test/new samples, by involving a training stage in the kNN classification. Specifically, in the training stage, kTree method first learns optimal k values for all training samples by a new sparse reconstruction model, and then constructs a decision tree (namely, kTree) using training samples and the learned optimal k values. In the test stage, the kTree fast outputs the optimal k value for each test sample, and then, the kNN classification can be conducted using the learned optimal k value and all training samples. As a result, the proposed kTree method has a similar running cost but higher classification accuracy, compared with traditional kNN methods, which assign a fixed k value to all test samples. Moreover, the proposed kTree method needs less running cost but achieves kNN methods. This actually reduces running cost of test stage. Finally, the experimental results on 20 real data sets showed that our proposed methods (i.e., kTree and k*Tree) are much more efficient than the compared methods in terms of classification tasks.

Index Terms—Decision tree, k nearest neighbor (kNN) classification, sparse coding.

I. Introduction

DIFFERENT from model-based methods which first learn models from training samples and then predict test samples with the learned model [1]–[6], the model-free k nearest neighbors (kNNs) method does not have training stage and conducts classification tasks by first calculating the distance between the test sample and all training samples to obtain its nearest neighbors and then conducting kNN classification[1] (which assigns the test samples with labels by the majority rule on the labels of selected nearest neighbors). Because

- and many other examples…find more here on Google Scholar: →



Code part

```python
from sklearn.neighbors import KNeighborsClassifier

iris_dataset = load_iris() # 3 classes, 4 features

X, y = iris_dataset.data, iris_dataset.target
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

X_train

```
array([[5.9, 3. , 4.2, 1.5],
       [5.8, 2.6, 4. , 1.2],
       [6.8, 3. , 5.5, 2.1],
       [4.7, 3.2, 1.3, 0.2],
       [6.9, 3.1, 5.1, 2.3],
       [5. , 3.5, 1.6, 0.6],
       [5.4, 3.7, 1.5, 0.2],
```

sepal length

sepal width

petal length

petal width

y_train

```
array([1, 1, 2, 0, 2, 0, 0, 1, 2, 2, 2, 2, 1, 2, 1, 1, 2, 2, 2, 2, 1, 2,
       1, 0, 2, 1, 1, 1, 1, 2, 0, 0, 2, 1, 0, 0, 1, 0, 2, 1, 0, 1, 2, 1,
       0, 2, 2, 2, 2, 0, 0, 2, 2, 0, 2, 0, 2, 2, 0, 0, 2, 0, 0, 0, 1, 2,
       2, 0, 0, 0, 1, 1, 0, 0, 1, 0, 2, 1, 2, 1, 0, 2, 0, 2, 0, 0, 2, 0,
       2, 1, 1, 1, 2, 2, 1, 1, 0, 1, 2, 2, 0, 1, 1, 1, 1, 0, 0, 0, 2, 1,
       2, 0])
```

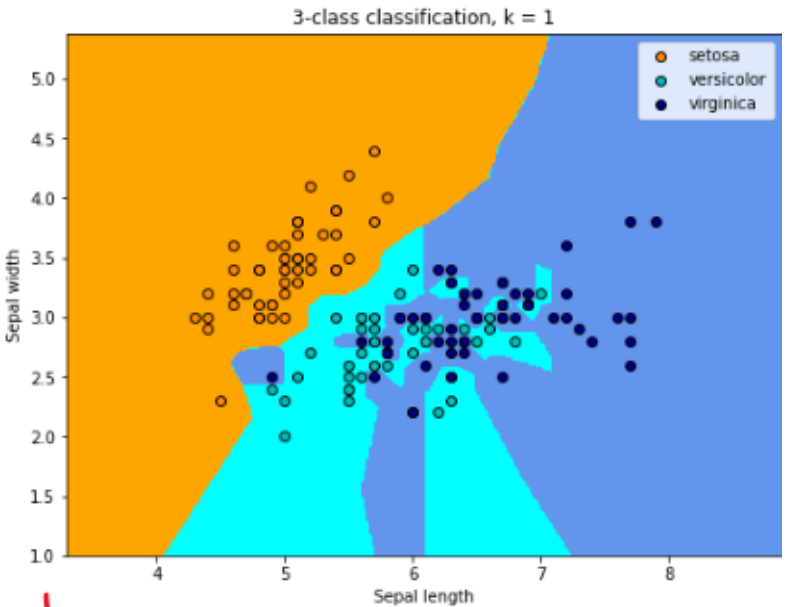
3 classes of flowers:

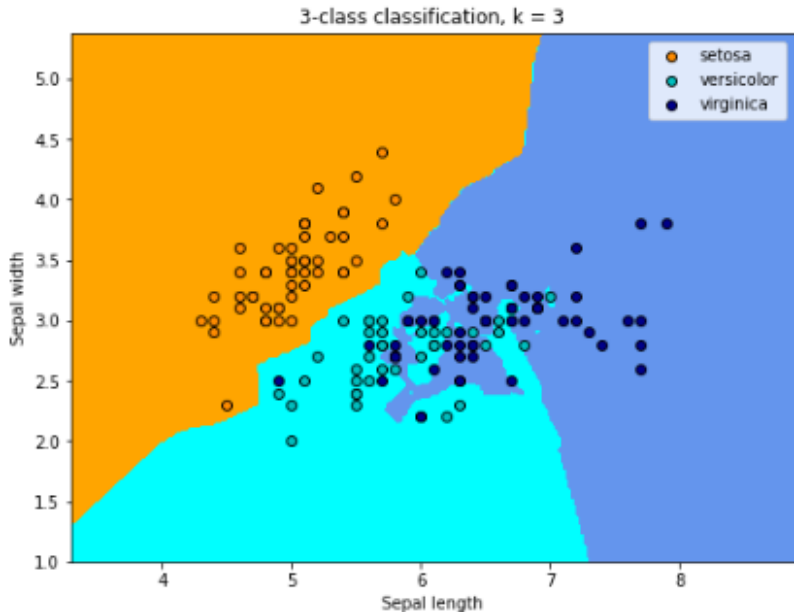
- setosa (0)
- versicolor (1)
- virginica (2)

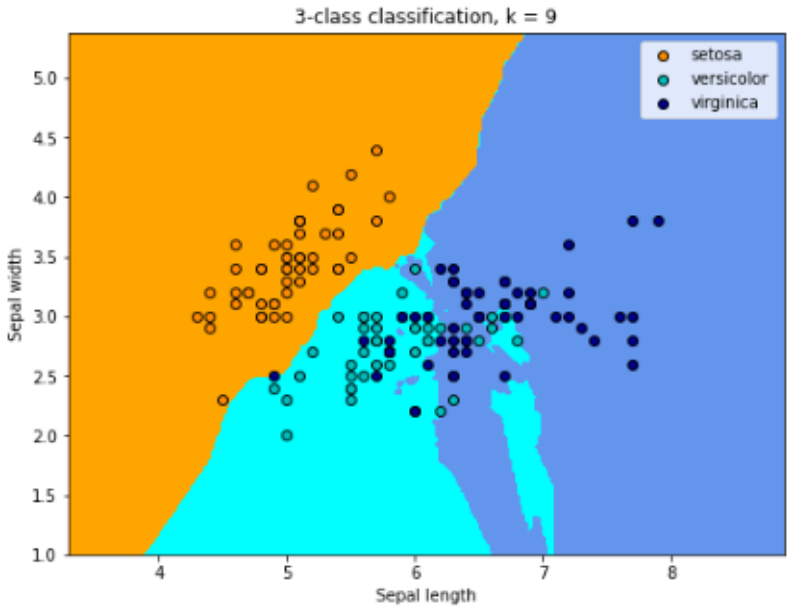
```python
clf = KNeighborsClassifier(n_neighbors=3)
clf.fit(X_train, y_train)

print("Predictions: ")
print(clf.predict(X_test))
print("Accuracy on the test set:")
print(clf.score(X_test, y_test))
```

```
Predictions:
[2 1 0 2 0 2 0 1 1 1 2 1 1 1 1 0 1 1 0 0 2 1 0 0 2 0 0 1 1 0 2 1 0 2 2 1 0
 2]
Accuracy on the test set:
0.9736842105263158
```







here we applied KNeighborsClassifier just for the first 2 features!
(because we cannot plot figures in 4D ☺)

2) Linear SVM classification

YouTube AT

andrew ng svm



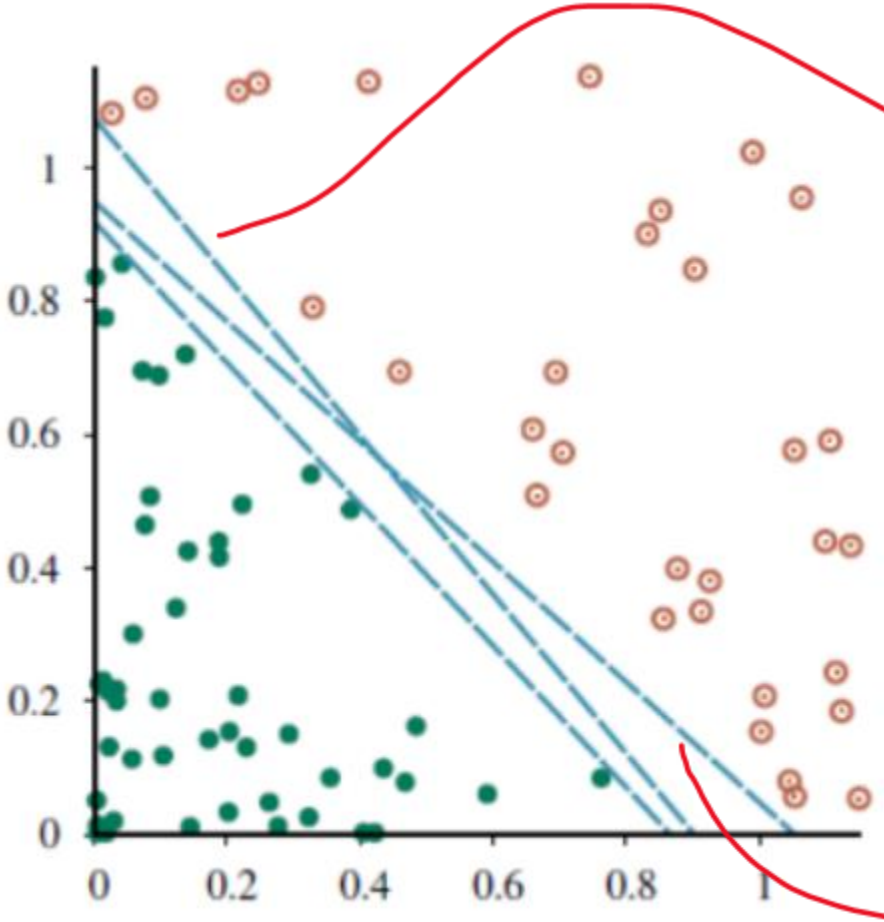
Support Vector Machines | ML-005 Lecture 12 | Stanford University | Andrew Ng

YouTube AT

statquest svm



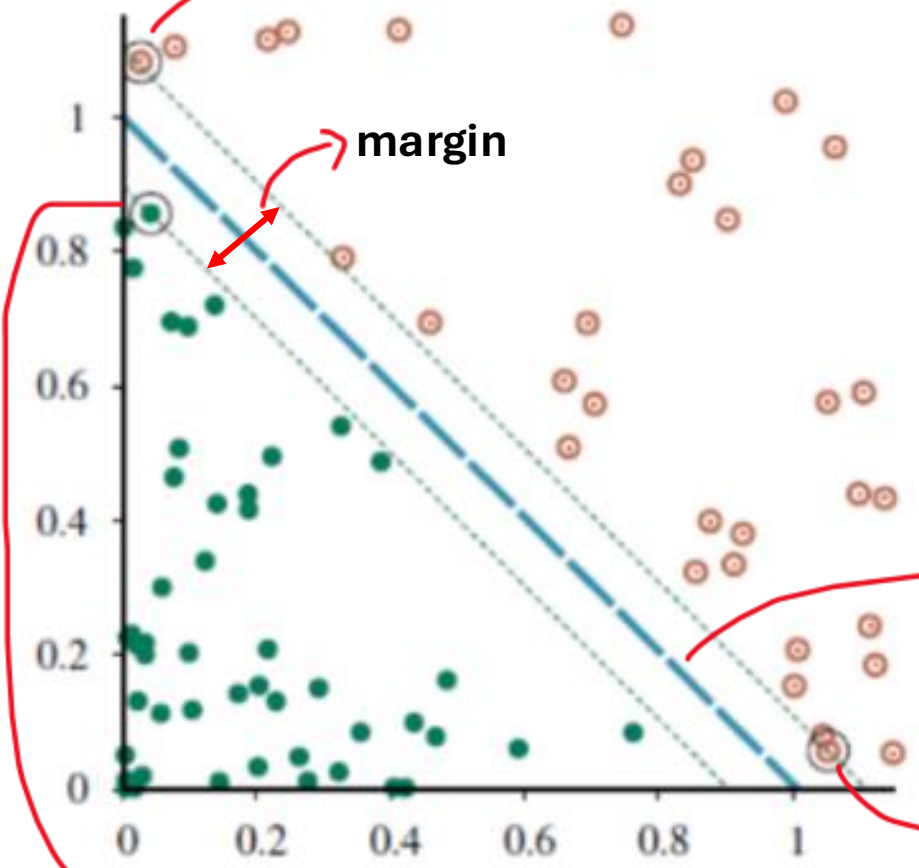
Support Vector Machines Part 1 (of 3): Main Ideas!!!



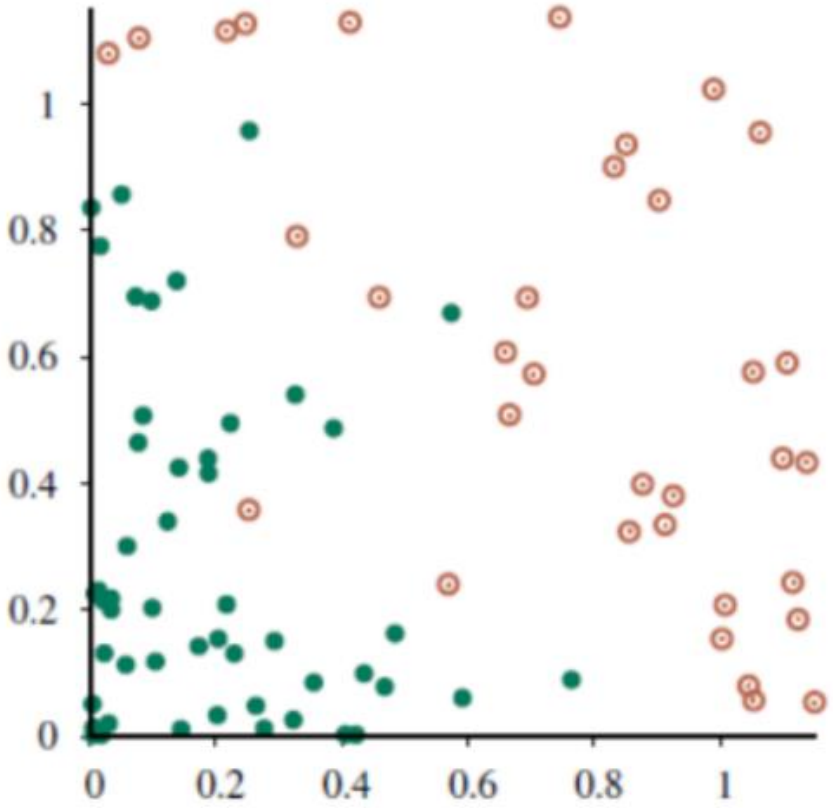
- We have the following dataset which is called **linearly separable** (the green points can be separated from the red points by a line)
- In the figure we have 3 possible **linear separators** (the blue lines)
- As we see, there can be infinitely many such linear separators (such lines)
- How do we know which one is best to choose?



We can shift the line **a tiny bit** up and down without coming into contact with any of examples



- To find a **unique solution**, we need to choose the line that maximizes the margin between the green and red circles
- Basically we want the blue line to be as far away from the closest training examples as possible (and obviously to separate the points ☺)
- This line is called **maximum margin separator**
- The points with large circles are called **support vectors**



- But what if the green and red points cannot be separated by a line (like in the left plot) ?

- In this case we want to find a line that doesn't perfectly separate the green and red points → we can misclassify a few points but better classifying the remaining ones

When the points
- can be separated by a line – **hard margin SVM**
- cannot be separated by a line – **soft margin SVM**

TL;DR — Intuiția din spatele SVM

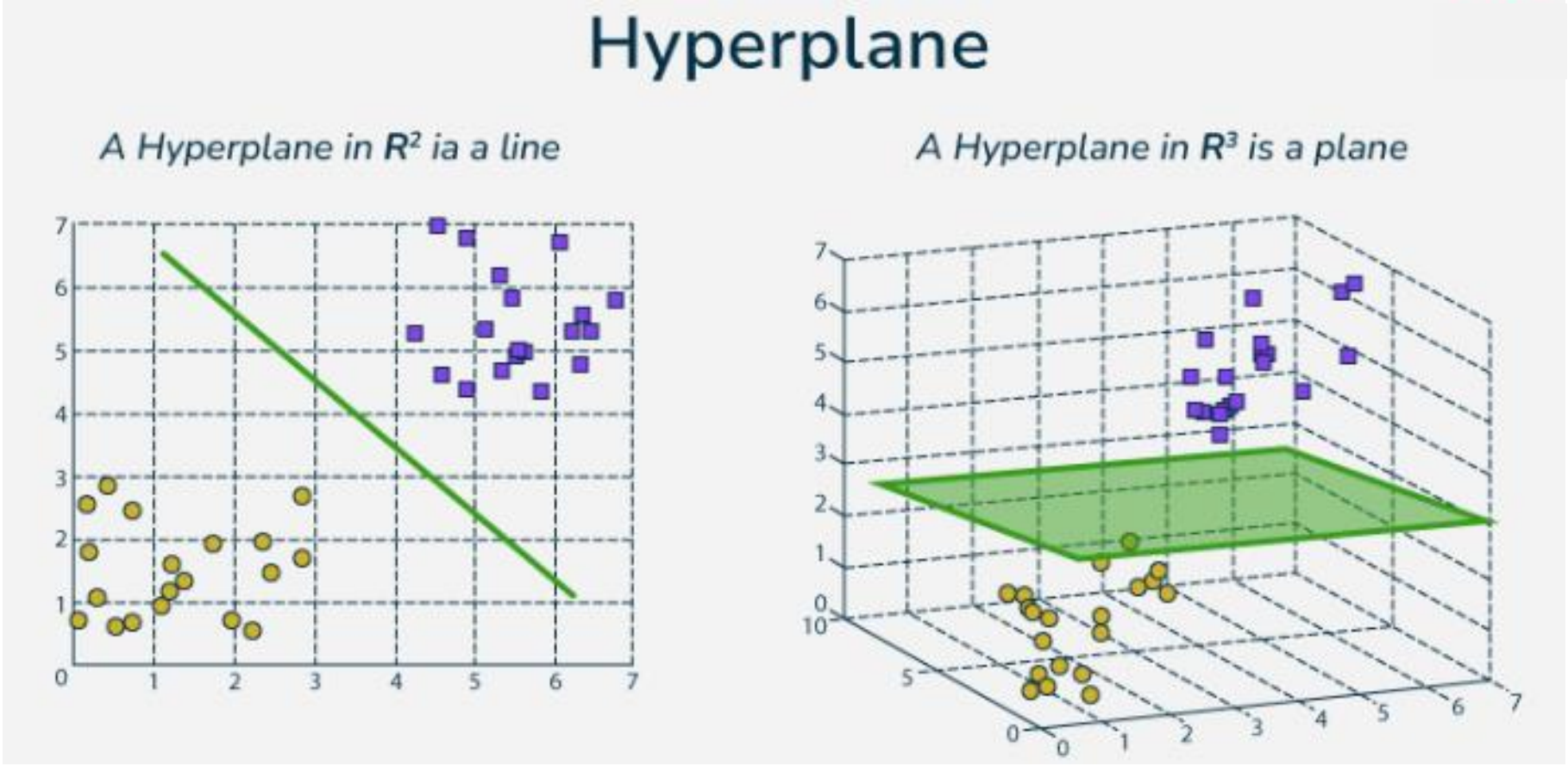
• **Marginea (margin):** Este **spațiul liber** dintre cele două clase — zona care separă punctele verzi de cele roșii. Cu cât marginea e mai mare, cu atât modelul e mai sigur și generalizează mai bine.

• **Support vectors:** Sunt **punctele aflate cel mai aproape de linia de separare**. Ele "țin" linia în poziție — dacă le miști, linia se schimbă! (De aici și numele: *"vectors that support the margin"*).

• **Scopul SVM:** Găsește **linia (sau planul)** care separă clasele și are **cea mai mare margine posibilă**, chiar dacă permite câteva clasificări greșite.

Optional – explaining the math behind

Hyperplane



In general, a **hyperplane** is defined by the following equation:

$$\{\boldsymbol{x} \in \mathbb{R}^d | \boldsymbol{w}^\top \boldsymbol{x} + w_0 = 0\}$$

$$\{\boldsymbol{x} \in \mathbb{R}^d | \boldsymbol{w}^\top \boldsymbol{x} + w_0 = 0\}$$

What this equation means:

1. $\mathbf{x} \in \mathbb{R}^d$:
    - This means that $\mathbf{x}$ is a data point in a **d-dimensional space** (where d is the number of features or variables). For example, in a 2D space, $\mathbf{x}$ would have two components like (x_1, x_2), and in 3D space, it would be (x_1, x_2, x_3), and so on.
2. $\mathbf{w}^T\mathbf{x}$:
    - This is a **dot product** between two vectors:
        - $\mathbf{w}$ is a vector of **weights** or **coefficients** that defines the orientation of the hyperplane in space.
        - $\mathbf{x}$ is the vector representing a data point.
    - The dot product measures the projection of $\mathbf{x}$ onto $\mathbf{w}$, which tells us how far and in which direction the point $\mathbf{x}$ is from the hyperplane. **NOTE 1**
3. w_0:
    - This is a **bias** or **intercept** term that shifts the hyperplane. It defines where the hyperplane is positioned relative to the origin of the space. **NOTE 2**

NOTE 1

- **Direction**: The **sign** of the dot product $\mathbf{w}^T\mathbf{x}$ tells us **which side of the hyperplane** the point $\mathbf{x}$ is on:
  - If $\mathbf{w}^T\mathbf{x} > 0$, then $\mathbf{x}$ lies on the side of the hyperplane that corresponds to one class (say, class +1).
  - If $\mathbf{w}^T\mathbf{x} < 0$, then $\mathbf{x}$ lies on the other side of the hyperplane (say, class -1).
- **Distance from the hyperplane**: The **magnitude** (absolute value) of the dot product tells us **how far** the point $\mathbf{x}$ is from the hyperplane. If the dot product is large, the point is far from the hyperplane. If it's small, the point is close to the hyperplane.

In summary, the dot product helps the SVM decide both **which side of the hyperplane** a point is on (classification) and **how far it is** from that boundary (confidence in the classification).

NOTE 2

- **Positioning the hyperplane**: The term w_0 shifts the hyperplane away from the origin. Without w_0, the hyperplane would always pass through the origin, which is not flexible enough for most classification problems.
  - **Positive** w_0 shifts the hyperplane in one direction away from the origin.
  - **Negative** w_0 shifts it in the opposite direction.

By adjusting w_0, the SVM can position the hyperplane at different places in the space, allowing for more flexibility in finding the optimal decision boundary that best separates the two classes.

- when training the classifier, we want to ensure that the examples with positive labels are on the positive side of the hyperplane:

$$\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0 \geq 0 \text{ when } y^{(i)} = +1,$$

  and the examples with negative labels are on the negative side:

$$\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0 < 0 \text{ when } y^{(i)} = -1,$$

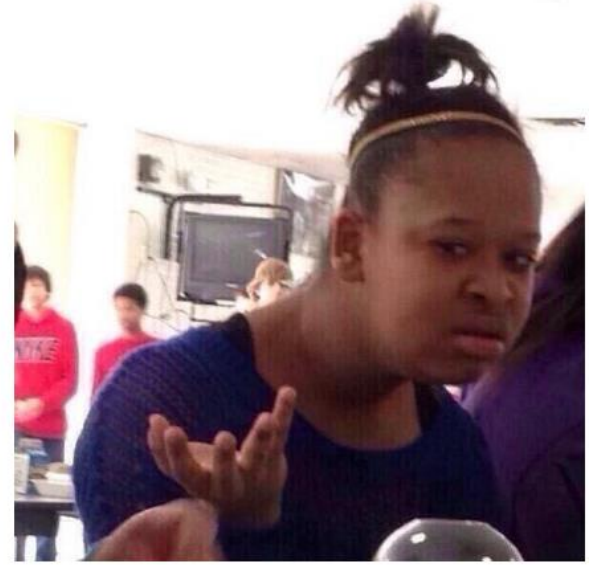
  for $1 \leq i \leq n$
- these two conditions are often presented in a single equation:

$$y^{(i)}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} + w_0) \geq 0.$$

- hard margin SVM optimization:

$$\min_{\mathbf{w},w_0} \frac{1}{2}\|w\|_2^2$$

subject to $y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + w_0) \geq 1,\ 1 \leq i \leq n.$



- soft margin SVM optimization:

$$\min_{\mathbf{w},w_0,\xi^{(1)},\ldots,\xi^{(n)}} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n}\xi^{(i)}$$

subject to $y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + w_0) \geq 1 - \xi^{(i)},\ \xi^{(i)} \geq 0,\ 1 \leq i \leq n.$

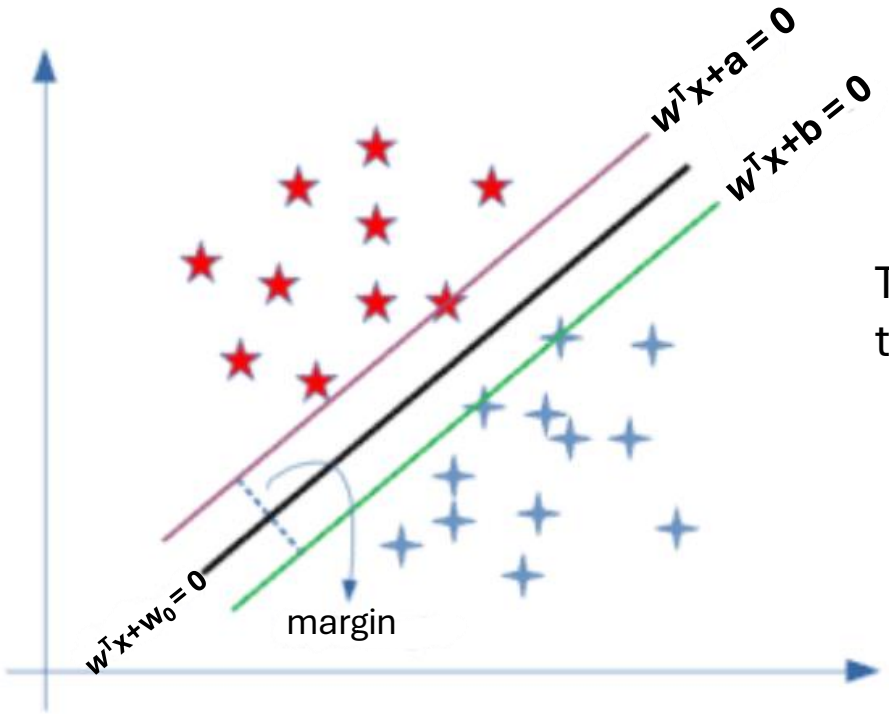
1. Hard Margin SVM Optimization:

$$\min_{\mathbf{w},w_0} \frac{1}{2}\|\mathbf{w}\|_2^2$$

Subject to:

$$y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + w_0\right) \geq 1, \quad 1 \leq i \leq n$$

Let's assume we have the following dataset:



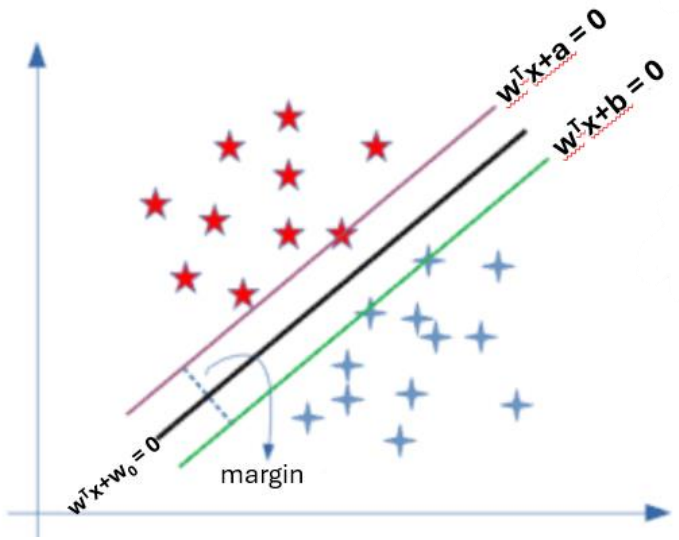
The hyperplane that separates the 2 classes is defined by:

$$w^Tx+w_0 = 0$$

Then, the 2 parallel hyperplanes that define the margin (purple and green line) are:

$$w^Tx+a = 0$$

$$w^Tx+b = 0$$



- Without allowing any misclassifications in the hard margin SVM, we want to maximize the distance between the two hyperplanes.
- To find this distance, we can use the formula for the distance of a point from a line. So the distance of the blue points and the red point from the black line would respectively be:

$$\frac{|\boldsymbol{w}^T\boldsymbol{x}+\mathrm{a}|}{||\boldsymbol{w}||_2} \quad \text{and} \quad \frac{|\boldsymbol{w}^T\boldsymbol{x}+\mathrm{b}|}{||\boldsymbol{w}||_2}$$

Distanța de la un punct la o dreaptă

Știm că distanța de la un punct la o dreaptă este lungimea dintre acel punct și piciorul perpendicularei de pe dreaptă dusă din acel punct. O formulă pentru a calcula această distanță va fi dată în cele ce urmează.

Fie $M(x_M, y_M)$ și dreapta $d_1 : ax + by + c = 0$.

1. *Distanța de la punctul M la dreapta d_1 este:*

$$d = \frac{|\, ax_M + by_M + c \,|}{\sqrt{a^2 + b^2}}.$$

2. *Dacă ecuația dreptei are altă formă, cum ar fi:*

$d_2 : y = mx + n$,

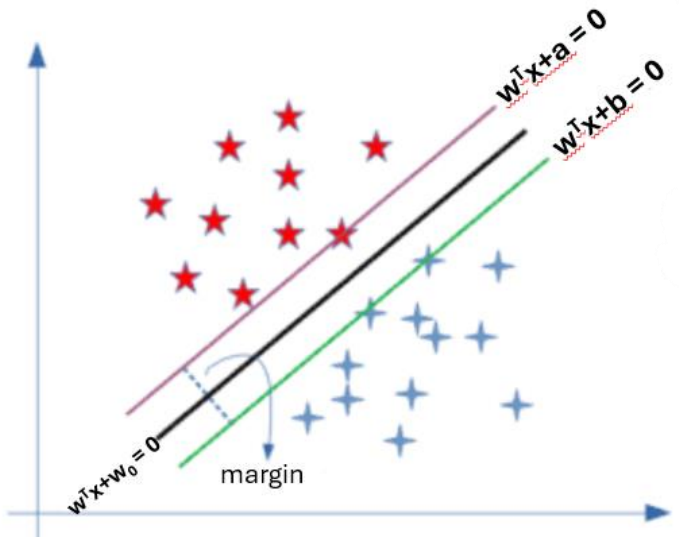
atunci distanța de la punctul M la dreapta d_2 este:

$$d = \frac{|\, mx_M - y_M + n \,|}{\sqrt{1 + m^2}}.$$

- As a result, the total margin would become:

$$\frac{|\,\mathrm{a}-\mathrm{b}\,|}{||\boldsymbol{w}||_2}$$

- We want to maximize this margin. Without the loss of generality, we can consider a = w_0-1 and b = w_0+1. Subsequently, the problem would be to maximize $\frac{2}{||\boldsymbol{w}||_2}$ or minimize $\frac{||\boldsymbol{w}||_2}{2}$



- To make the problem easier when taking the gradients, we'll instead work with its squared form:

$$\min_{\boldsymbol{w}, w_0} \frac{1}{2}||\boldsymbol{w}||_2^2 \quad (1)$$

- This optimization comes with some constraints. Let's assume that the labels for our classes are {-1, +1} (the red stars are +1 and blue stars are -1).

- When classifying the data points, we want the points belonging to positives classes to be greater than +1 and the points belonging to the negative classes to be less than -1, meaning:

  - $\boldsymbol{w^Tx+a \geq 0 \Leftrightarrow w^Tx+w_0-1 \geq 0 \Leftrightarrow w^Tx+w_0 \geq 1}$
  - $\boldsymbol{w^Tx+b \leq 0 \Leftrightarrow w^Tx+w_0+1 \leq 0 \Leftrightarrow w^Tx+w_0 \leq -1}$

$$y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + w_0) \geq 1, \ 1 \leq i \leq n \quad (2)$$

(1), (2) →

$$\min_{\mathbf{w}, w_0} \frac{1}{2}\|w\|_2^2$$
$$\text{subject to } y^{(i)}(\mathbf{w}^T\mathbf{x}^{(i)} + w_0) \geq 1, \ 1 \leq i \leq n.$$



2. Soft Margin SVM Optimization:

$$\min_{\mathbf{w},w_0,\xi^{(1)},\ldots,\xi^{(n)}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\xi^{(i)}$$

Subject to:

$$y^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + w_0\right) \geq 1 - \xi^{(i)}, \quad \xi^{(i)} \geq 0, \quad 1 \leq i \leq n$$



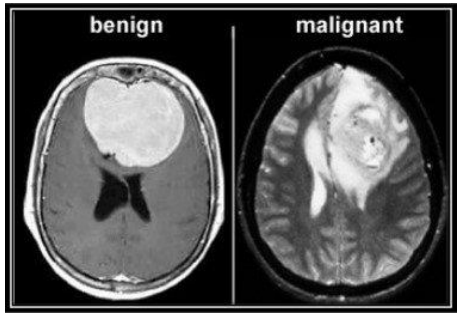
In a similar way as Hard Margin SVM Optimization but more numbers and Greek notations, for more info see the following link:

https://www.baeldung.com/cs/svm-hard-margin-vs-soft-margin

Real-World Uses of Support Vector Machines (SVM)

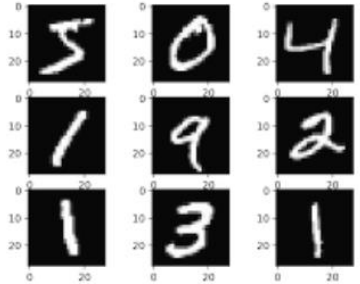
Where SVM is used (some examples):

- **Medical diagnosis:** Classifying tumors (benign vs. malignant), disease detection from MRI or gene data.



- **Computer vision:** Object / face recognition, handwriting classification (MNIST dataset).



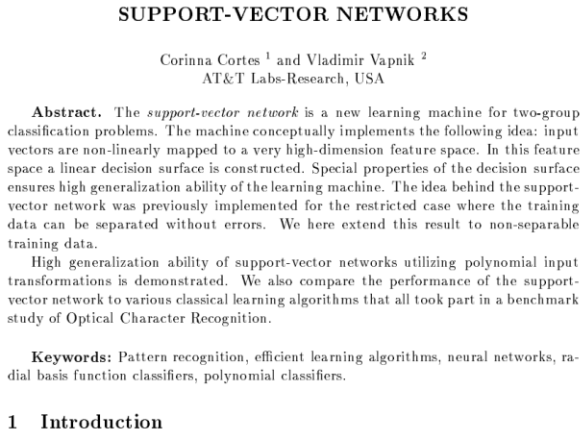


- **Fraud detection:** Detect unusual transaction patterns indicating fraud.



Notable papers

- Cortes & Vapnik (1995) — **Support-vector networks** → the original SVM paper.



SUPPORT-VECTOR NETWORKS

Corinna Cortes [1] and Vladimir Vapnik [2]
AT&T Labs-Research, USA

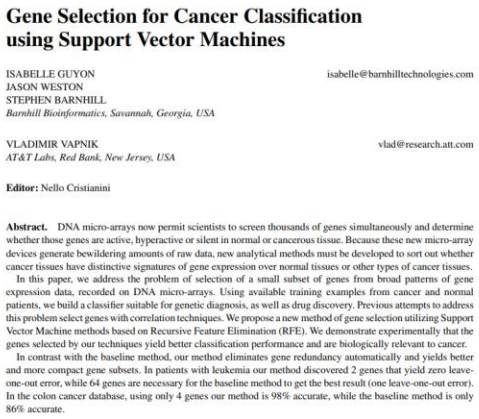
Abstract. The *support-vector network* is a new learning machine for two-group classification problems. The machine conceptually implements the following idea: input vectors are non-linearly mapped to a very high-dimension feature space. In this feature space a linear decision surface is constructed. Special properties of the decision surface ensures high generalization ability of the learning machine. The idea behind the support-vector network was previously implemented for the restricted case where the training data can be separated without errors. We here extend this result to non-separable training data.

High generalization ability of support-vector networks utilizing polynomial input transformations is demonstrated. We also compare the performance of the support-vector network to various classical learning algorithms that all took part in a benchmark study of Optical Character Recognition.

Keywords: Pattern recognition, efficient learning algorithms, neural networks, radial basis function classifiers, polynomial classifiers.

1 Introduction

- Guyon et al. (2002)— **Gene selection** for **cancer classification using support vector machines**



Gene Selection for Cancer Classification using Support Vector Machines

ISABELLE GUYON isabelle@barnhilltechnologies.com
JASON WESTON
STEPHEN BARNHILL
Barnhill Bioinformatics, Savannah, Georgia, USA

VLADIMIR VAPNIK vlad@research.att.com
AT&T Labs, Red Bank, New Jersey, USA

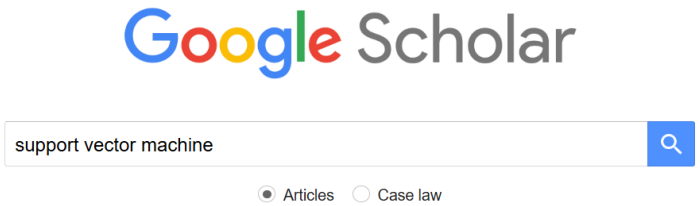
Editor: Nello Cristianini

Abstract. DNA micro-arrays now permit scientists to screen thousands of genes simultaneously and determine whether those genes are active, hyperactive or silent in normal or cancerous tissue. Because these new micro-array devices generate bewildering amounts of raw data, new analytical methods must be developed to sort out whether cancer tissues have distinctive signatures of gene expression over normal tissues or other types of cancer tissues.

In this paper, we address the problem of selection of a small subset of genes from broad patterns of gene expression data, recorded on DNA micro-arrays. Using available training examples from cancer and normal patients, we build a classifier suitable for genetic diagnosis, as well as drug discovery. Previous attempts to address this problem select genes with correlation techniques. We propose a new method of gene selection utilizing Support Vector Machine methods based on Recursive Feature Elimination (RFE). We demonstrate experimentally that the genes selected by our techniques yield better classification performance and are biologically relevant to cancer.

In contrast with the baseline method, our method eliminates gene redundancy automatically and yields better and more compact gene subsets. In patients with leukemia our method discovered 2 genes that yield zero leave-one-out error, while 64 genes are necessary for the baseline method to get the best result (one leave-one-out error). In the colon cancer database, using only 4 genes our method is 98% accurate, while the baseline method is only 86% accurate.

- and many other examples…find more here on Google Scholar: →



Code part

Let’s build an SVM classifier to distinguish between the setosa and versicolor types in the iris dataset, based only on the petal length and petal width features:

```
X = iris_dataset["data"][:, (2, 3)]  # petal length, petal width
y = iris_dataset["target"]

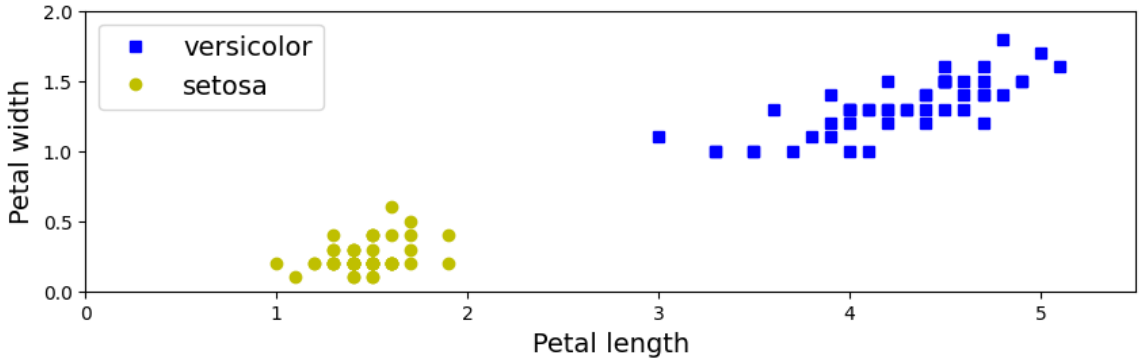
setosa_or_versicolor = (y == 0) | (y == 1)
X = X[setosa_or_versicolor]
y = y[setosa_or_versicolor]
```

```
y
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

```
X
```

```
array([[1.4, 0.2],
       [1.4, 0.2],
       [1.3, 0.2],
       [1.5, 0.2],
       [1.4, 0.2],
       [1.7, 0.4],
       [1.4, 0.3],
       [1.5, 0.2],
       [1.4, 0.2],
```



1) Hard margin SVM

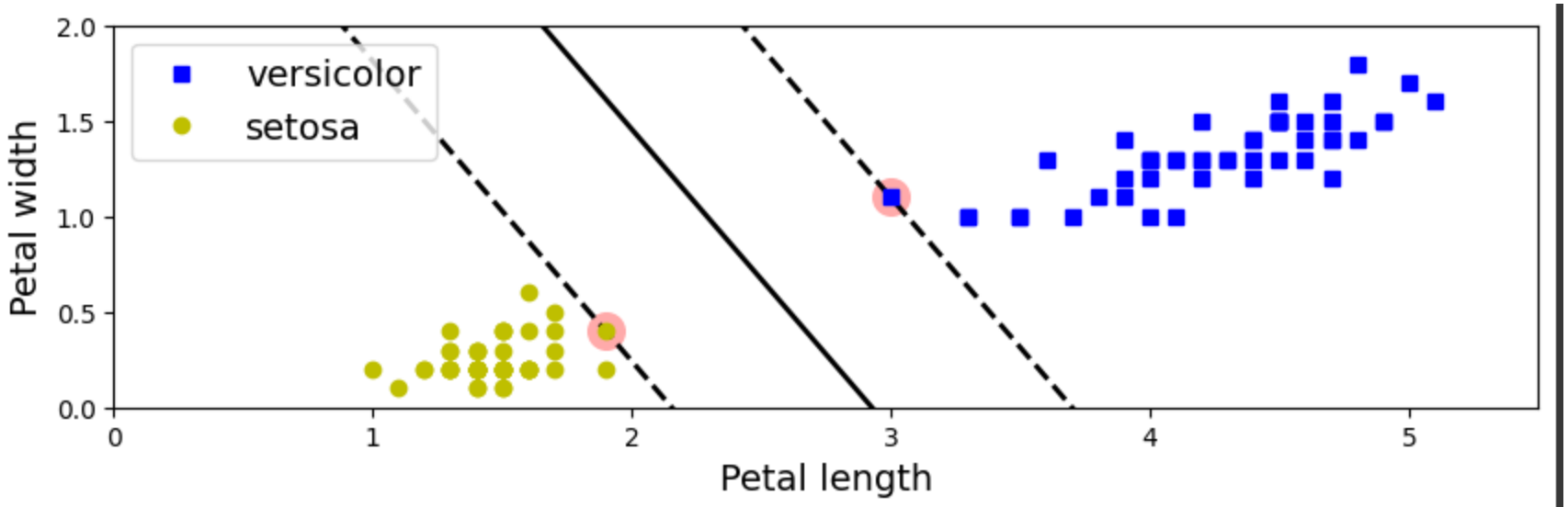
this means the decision boundary is a linear hyperplane (a line, a plane, etc.) ☺

```
from sklearn.svm import SVC

svm_clf = SVC(kernel="linear", C=float("inf"))
svm_clf.fit(X, y)
```

- for hard margin **C** is set to $+\infty$
- it means that we want a perfect separation of the data without allowing any misclassifications

function for plot already done in lab



1) Soft margin SVM

- a small value of **C** makes the SVM more tolerant of misclassifications and margin violations. The model will allow more points to be within the margin or even on the wrong side of the hyperplane
- a larger **C** value forces the SVM to be stricter

```
from sklearn.svm import LinearSVC

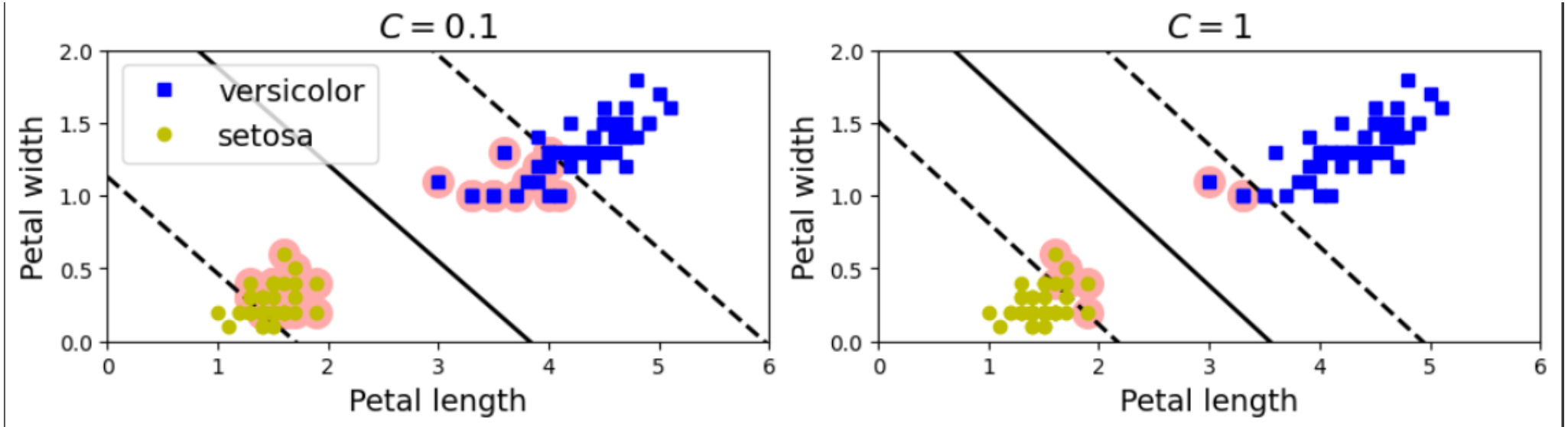
svm_clf1 = LinearSVC(C=0.1, loss="hinge", random_state=42)
svm_clf2 = LinearSVC(C=1, loss="hinge", random_state=42)

svm_clf1.fit(X, y)
svm_clf2.fit(X, y)
```

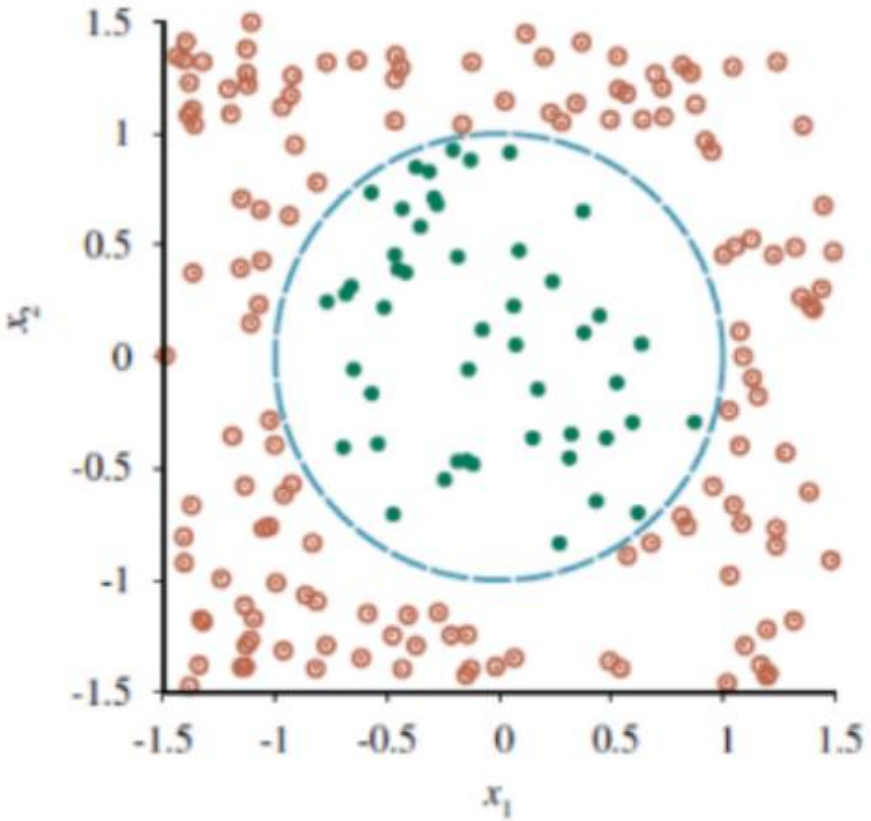
is used in SVM to calculate the error for misclassified points:

- if the point is correctly classified, the loss is 0
- if the point is misclassified, the loss is proportional to how far the point is from being correctly classified

function for plot already done in lab

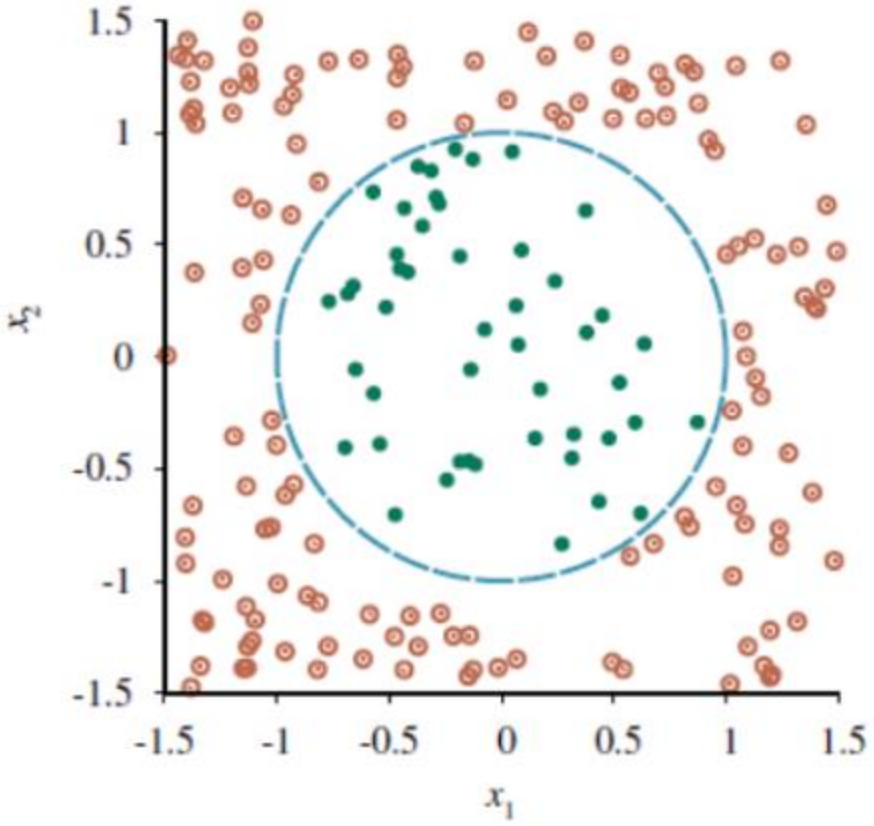


3) Nonlinear SVM classification

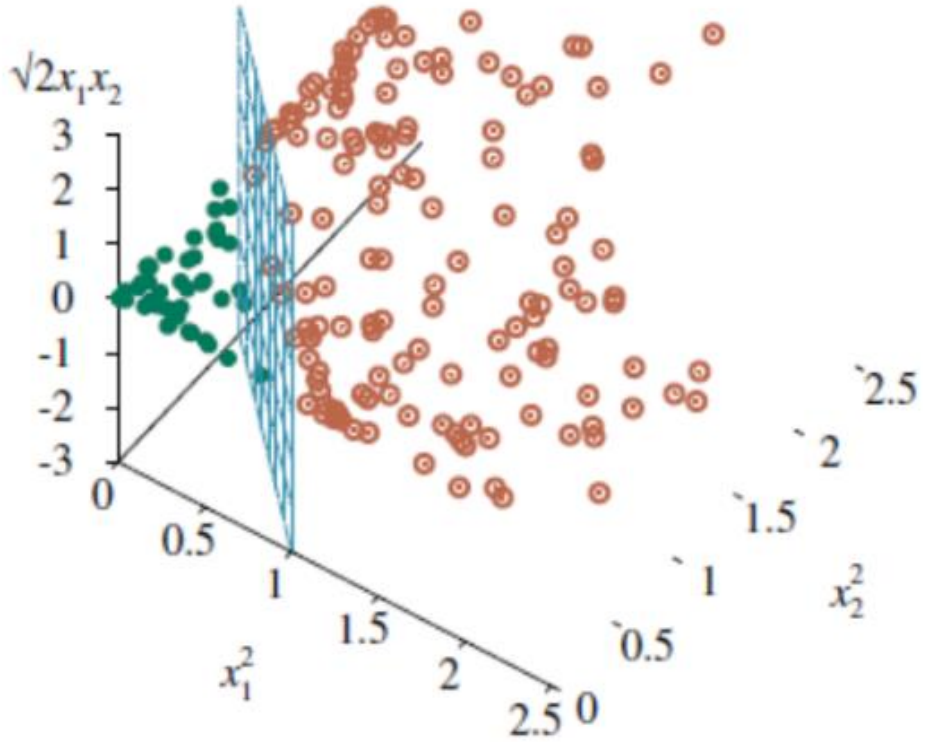


- What happens if we have nonlinear datasets like in the left plot?
- **SOLUTION**: add more features (e.g polynomial features) such that to create a linearly separable dataset in higher dimensions

Nonlinearly separable



Linearly separable



$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

NOTE: when we say **linearly** or **nonlinearly** we are not necessarily referring to a line, we refer to something that is **linear** (the equation of a plane in 3D is also linear)

$$\boldsymbol{x} = \begin{bmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2}x_1x_2 \end{bmatrix}$$

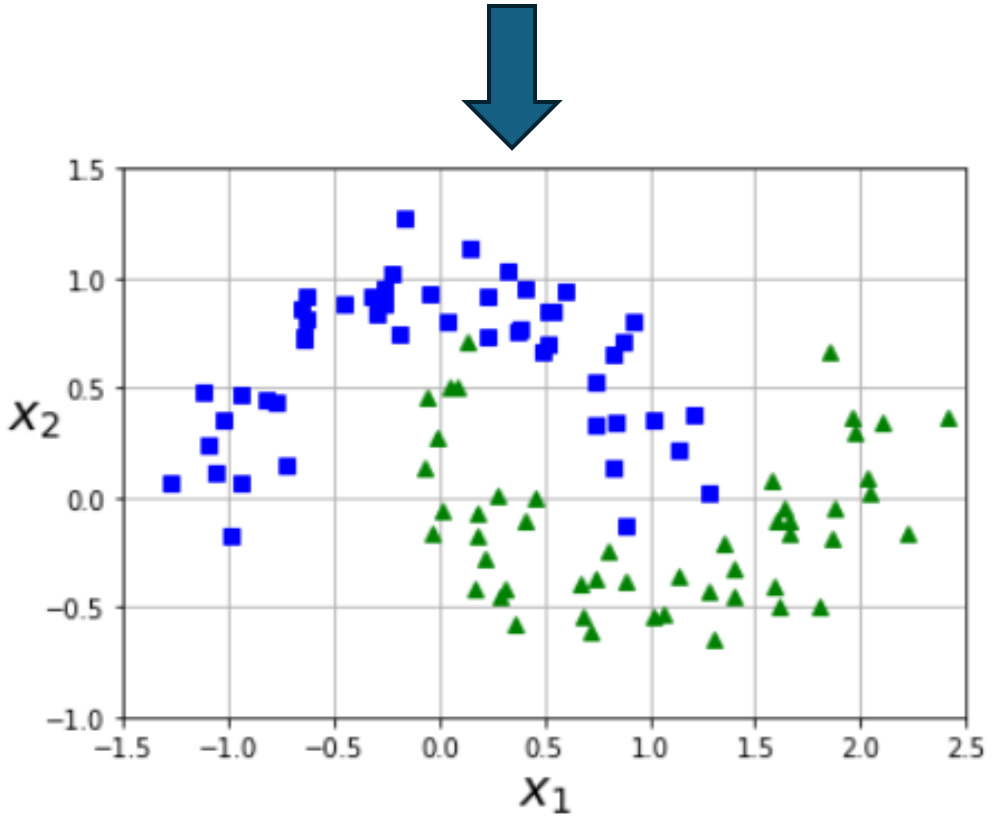
The generalization of a 2D line or a 3D plane in called **hyperplane**

Code part

```python
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)

def plot_dataset(X, y, axes):
    plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
    plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^")
    plt.axis(axes)
    plt.grid(True, which='both')
    plt.xlabel(r"$x_1$", fontsize=20)
    plt.ylabel(r"$x_2$", fontsize=20, rotation=0)

plot_dataset(X, y, [-1.5, 2.5, -1, 1.5])
plt.show()
```



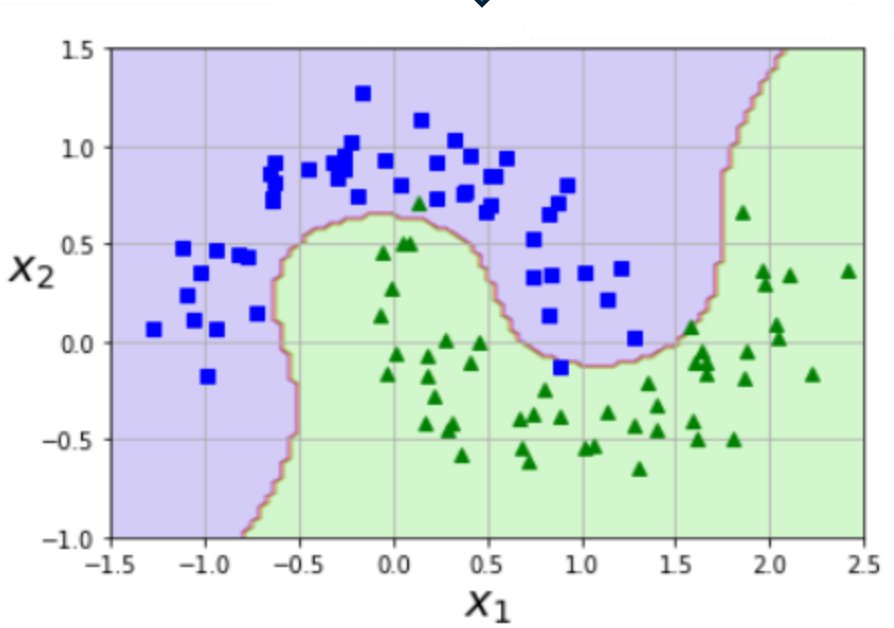
our dataset

```python
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

polynomial_svm_clf = Pipeline([
        ("poly_features", PolynomialFeatures(degree=3)),
        ("scaler", StandardScaler()),
        ("svm_clf", LinearSVC(C=10, loss="hinge", random_state=42))
    ])

polynomial_svm_clf.fit(X, y)
```

function for plot already done in lab



You don’t need to worry about them, You will have these parameters for test ☺

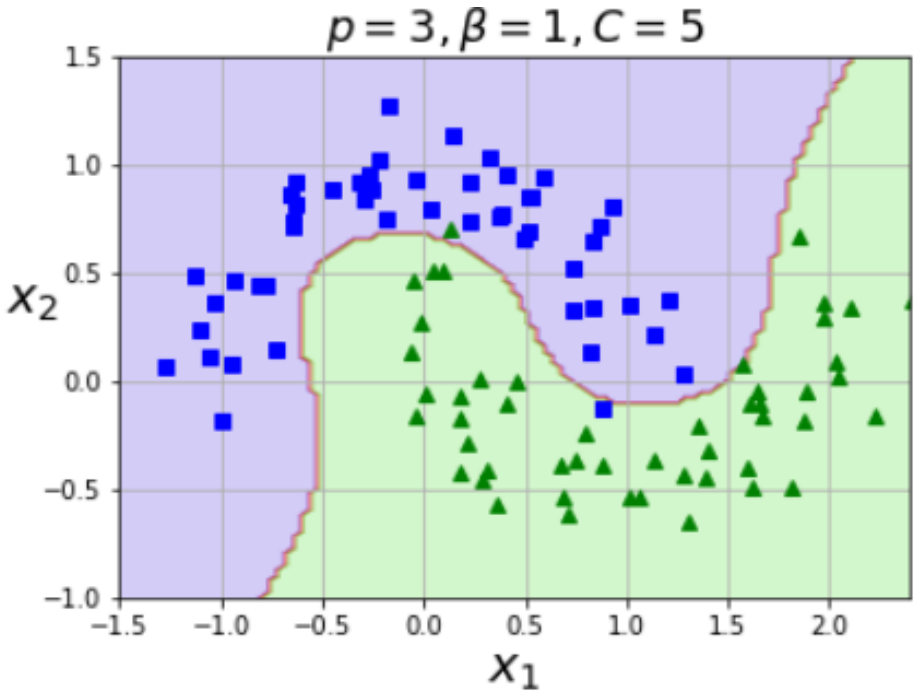
....or alternatively using the **kernel trick**...

don't worry about them

```python
from sklearn.svm import SVC

poly_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
    ])
poly_kernel_svm_clf.fit(X, y)
```

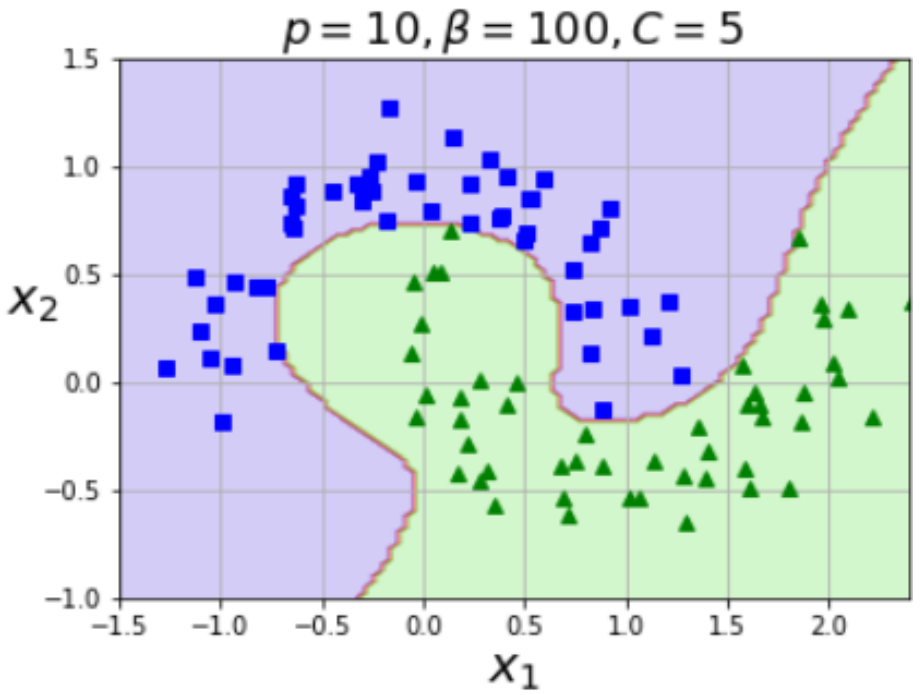
function for plot already done in lab



don't worry about them

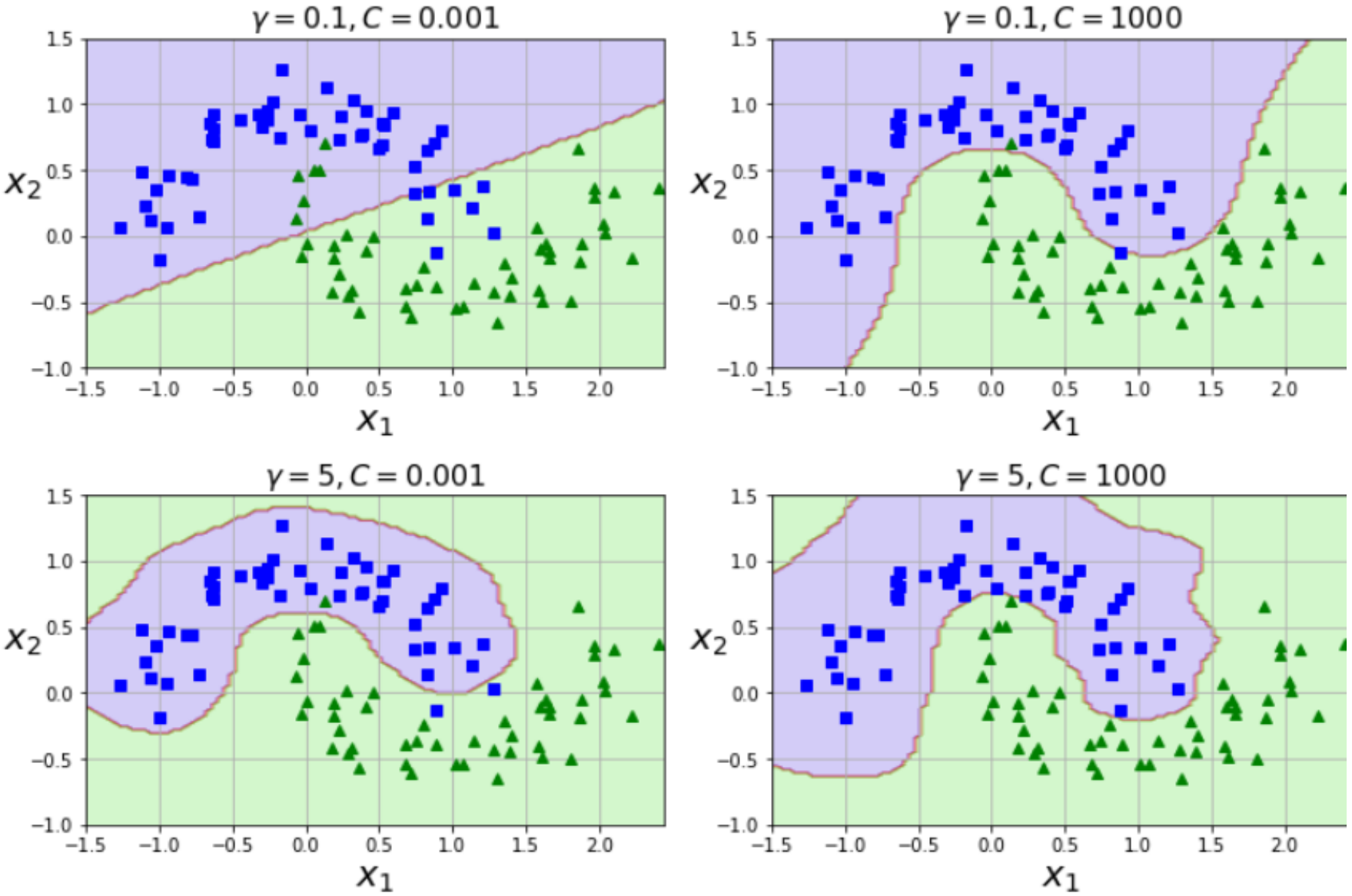
```python
poly100_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="poly", degree=10, coef0=100, C=5))
    ])
poly100_kernel_svm_clf.fit(X, y)
```

function for plot already done in lab

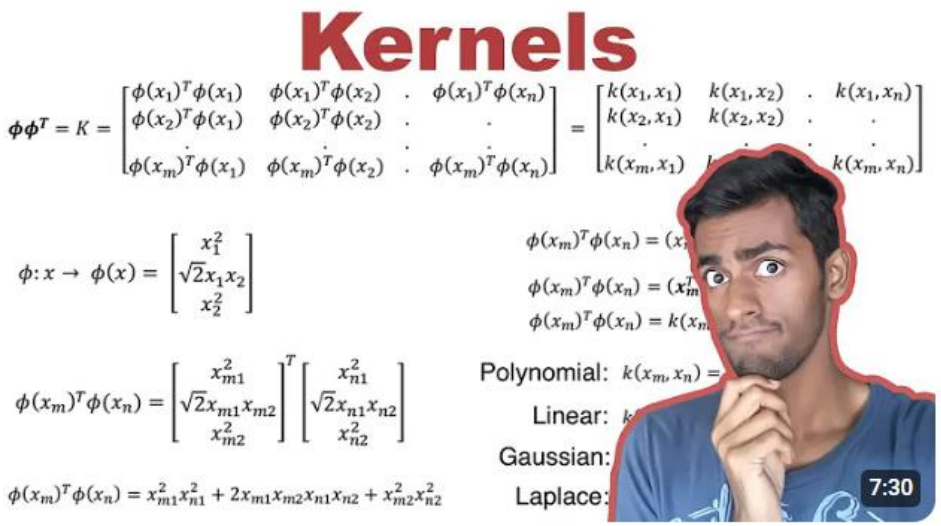


...or using more sophisticated kernels...





What is **kernel trick**?



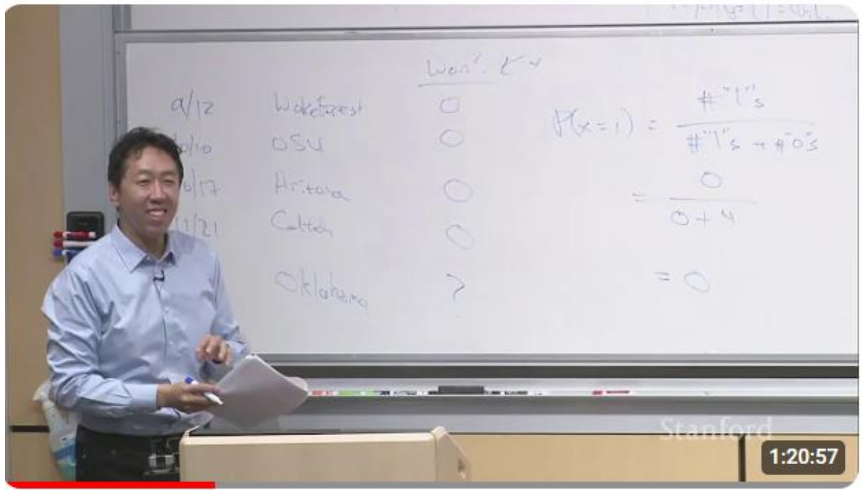
The Kernel Trick - THE MATH YOU SHOULD KNOW!

175 K de vizionări • acum 6 ani

CodeEmporium

Some parametric methods, like polynomial regression and **Support Vector Mach**

Capitole: 5 Intro | Kernel Matrix | Simplification | The Kernel Trick | |



Lecture 6 - Support Vector Machines | Stanford CS229: Machine Learning Andrew Ng (Autumn 2018)

241 K de vizionări • acum 4 ani

Stanford Online

For more information about Stanford's Artificial Intelligence professional and graduate programs, visit: https://stanford.io/ai

Subtitrări

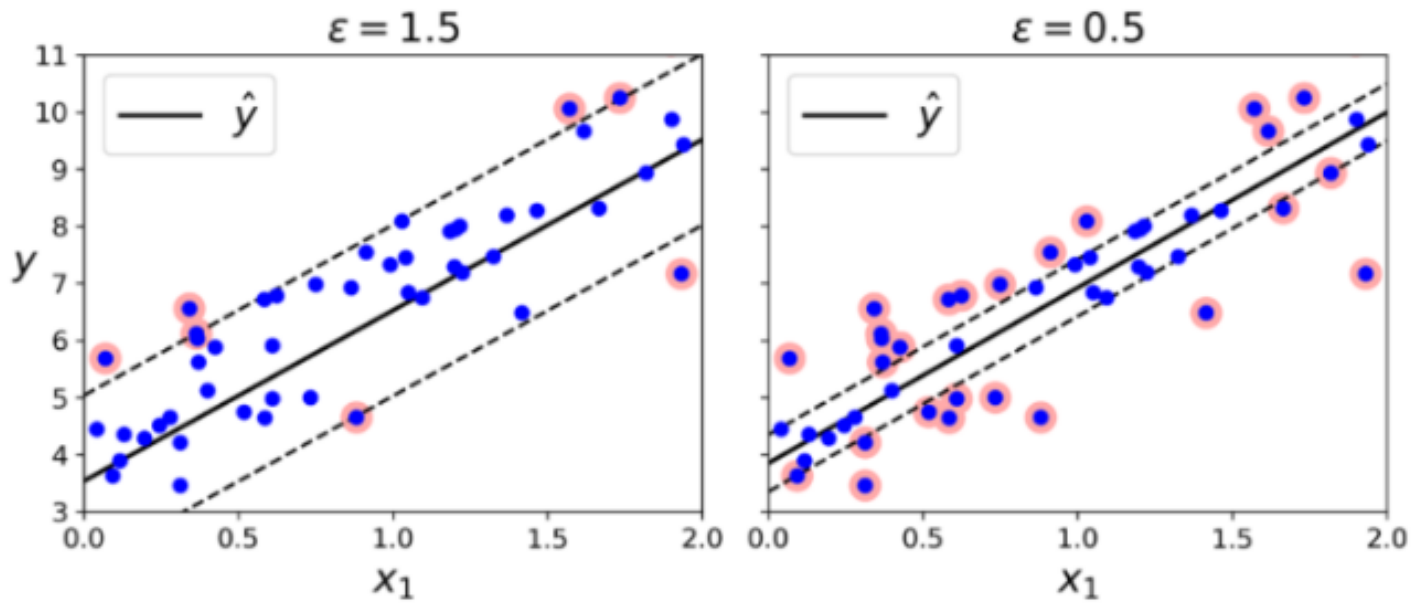
Momente: 16 Advice for Applying Machine Learning Algorithms | Recap | Build a Generative Model | Laplace...

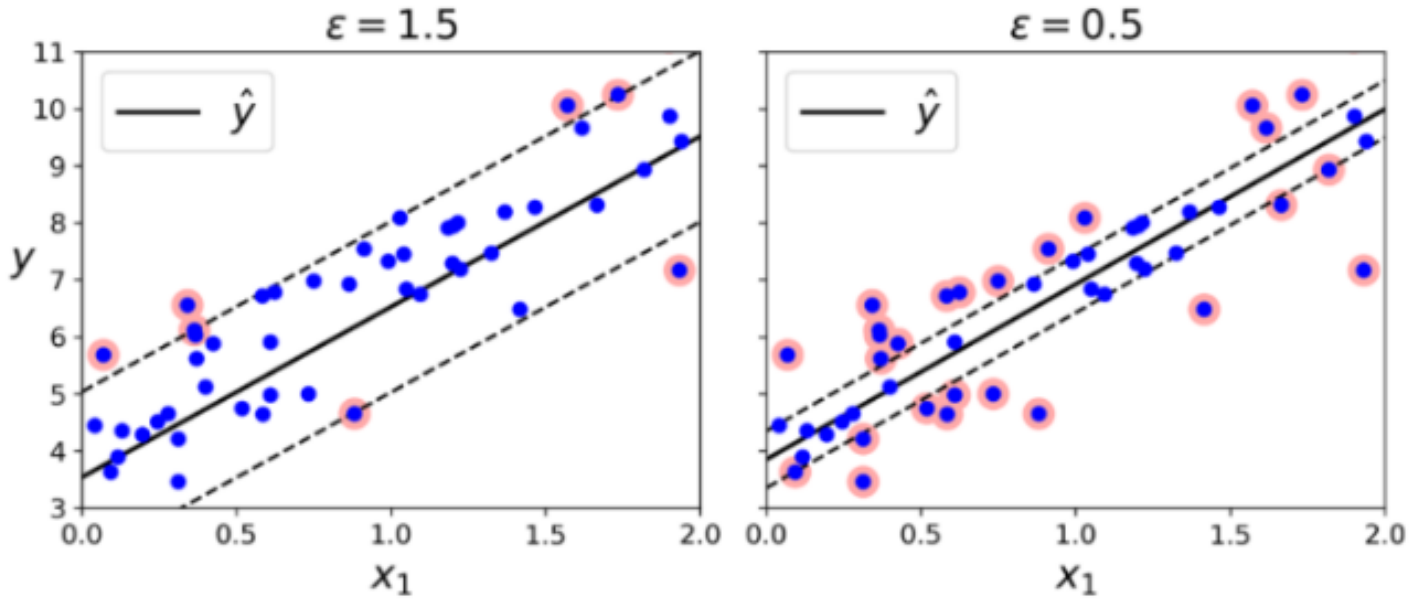
4) SVM regression

Reversing the Objective from Classification to Regression:

- In SVM **classification**, the goal is to find the largest possible margin between two classes, while minimizing errors (i.e., correctly classifying points).
- For **SVM regression (SVR)**, the goal is a bit different. Instead of finding a margin between classes, SVR tries to **fit a function that is as flat as possible, with the most data points falling inside a margin around it**. In simple terms, SVR attempts to make predictions within a specific "tube" or margin, while limiting the number of points that fall outside this tube.



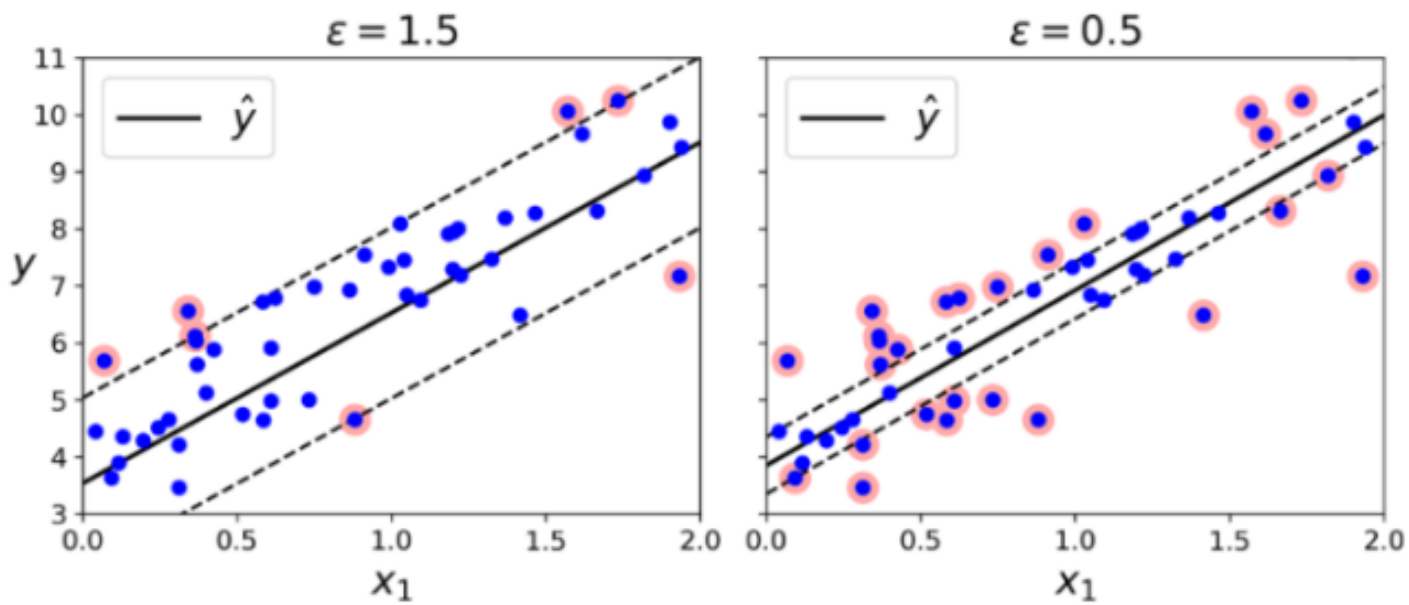




- The **solid line** in both graphs represents the predicted values by the SVR model.
- The **dotted lines** represent the boundaries of the margin (tube).
- The **points inside the margin** are considered within the acceptable range (no errors).
- The **circled points outside the margin** are the margin violations (errors), where the model's prediction is too far from the actual value.

The Role of the Hyperparameter ϵ (Epsilon):

- The hyperparameter ϵ defines the width of the margin (the "tube"). It controls how much deviation from the true values is tolerated.
- If ϵ is large (as in the graph on the left with $\epsilon = 1.5$), a wider margin is allowed, meaning more data points can fall inside the tube.
- If ϵ is small (as in the graph on the right with $\epsilon = 0.5$), the margin is narrower, and fewer points will fit inside the tube, which means the model has to be more precise in its predictions.



TL;DR — Intuiția din spatele SVM Regression (SVR)

- În **clasificare**, SVM caută **linia care separă cel mai bine două clase**, lăsând un spațiu liber (margine) între ele.

- În **regresie**, nu mai vrem să separăm clase, ci să **desenăm o linie care urmează cât mai bine trendul datelor**.

- În jurul acestei linii, SVR creează o **zonă-tub (ε)**:
  → punctele din interiorul tubului sunt considerate **„predicții bune”**,
  → iar cele din afară sunt **erori** (modelul nu le poate explica perfect).

- **Epsilon (ε)** controlează cât de „larg” e acest tub:
  - **ε mare:** modelul e mai tolerant, trece „printre” date fără să le urmeze prea exact.
  - **ε mic:** modelul devine mai strict, încearcă să treacă aproape prin fiecare punct.

Code part

linear SVM regression:

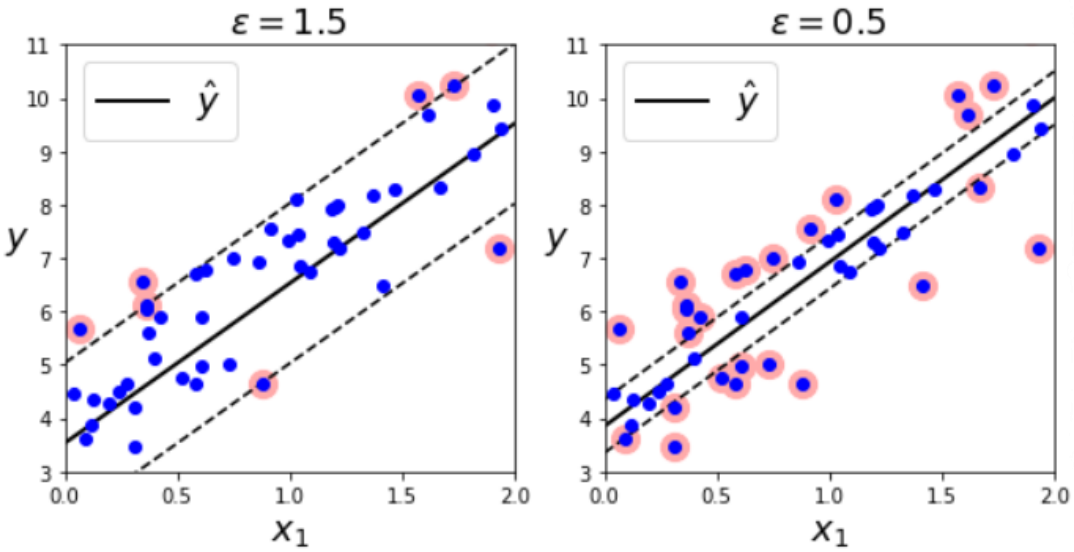
```python
np.random.seed(42)
n = 50
X = 2 * np.random.rand(n, 1)
y = (4 + 3 * X + np.random.randn(n, 1)).ravel()

from sklearn.svm import LinearSVR

svm_reg1 = LinearSVR(epsilon=1.5, random_state=42)
svm_reg2 = LinearSVR(epsilon=0.5, random_state=42)
svm_reg1.fit(X, y)
svm_reg2.fit(X, y)
```

You don’t need to worry about them, you will have these parameters for test ☺

function for plot already done in lab



nonlinear SVM regression:

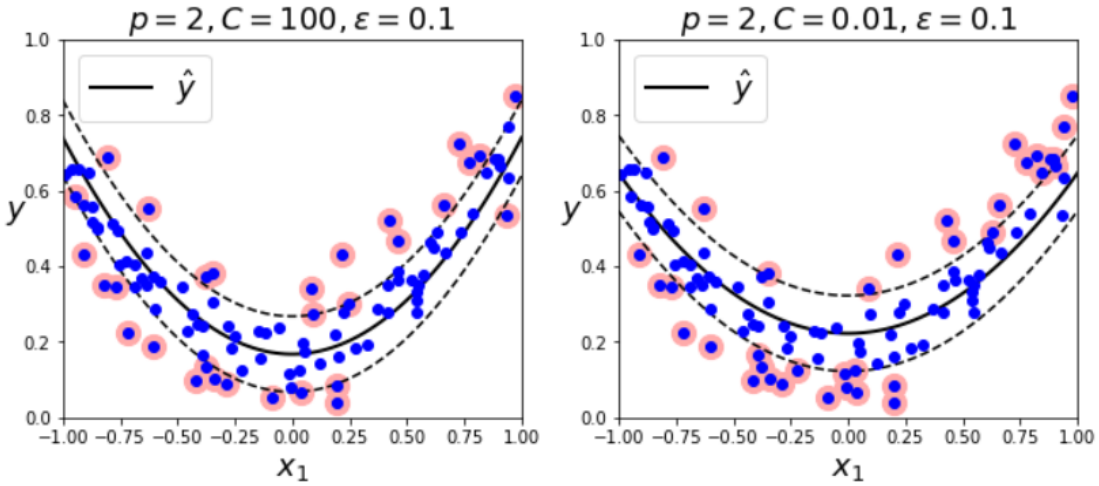
```python
np.random.seed(42)
n = 100
X = 2 * np.random.rand(n, 1) - 1
y = (0.2 + 0.1 * X + 0.5 * X**2 + np.random.randn(n, 1)/10).ravel()

from sklearn.svm import SVR

svm_poly_reg1 = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
svm_poly_reg2 = SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1)
svm_poly_reg1.fit(X, y)
svm_poly_reg2.fit(X, y)
```

You don’t need to worry about them, you will have these parameters for test ☺

function for plot already done in lab





- **k-Nearest Neighbors (k-NN):** Looks at the *closest data points* and predicts based on their values:
  - *Classification:* Chooses the most common label among neighbors.
  - *Regression:* Averages their numeric values to make a prediction.

- **Linear SVM (Classification):** Finds the *best straight line (or plane)* that separates classes with the *largest margin* between them.

- **Nonlinear SVM (Classification):** Uses the *kernel trick* to bend the space, allowing separation when data isn't linearly separable.

- **SVM Regression (Linear & Nonlinear):** Tries to fit a line or curve that stays *as close as possible to all points*, but *ignores small errors* within a tolerance range (the "ε-tube").

Exercises

Laboratory 3 – Exercises

1. Apply the k-nearest neighbors algorithm on the `moons` dataset. The number of samples of this dataset is 200. Split the data such that the test split is 45% of the initial dataset. Set the number of neighbors parameter to 2 and compute the accuracy on the test set. Visualize the decision boundary of the KNN classifiers using different values for k = $\{1, 5, 9\}$.

2. Use the following dataset:

```
X1, y1 = make_gaussian_quantiles(cov=7.,
                                 n_samples=250, n_features=2,
                                 n_classes=3, random_state=1),
```

in order to apply the nonlinear SVM classification. Use the polynomial kernel and set the `degree`, `coef0`, and `C` parameters to 3, 10, and 5 respectively. Plot the decision boundary of the nonlinear SVM classification.

3. Apply linear SVM regression to the following dataset:

```
X = 6 * np.random.rand(n, 1)
y = (1 - 2 * X + np.random.randn(n, 1)).ravel().
```

Set the width of the margin to 0.8 and plot the fit line.

Solutions

1. Apply the k-nearest neighbors algorithm on the **moons** dataset. The number of samples of this dataset is 200. Split the data such that the test split is 45% of the initial dataset. Set the number of neighbors parameter to 2 and compute the accuracy on the test set. Visualize the decision boundary of the KNN classifiers using different values for $k = \{1, 5, 9\}$.

```python
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Generating the moons dataset with 200 samples and a random state for reproducibility
X, y = make_moons(n_samples=200, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.45 ,random_state=0)

clf = KNeighborsClassifier(n_neighbors=2)
clf.fit(X_train, y_train)

# Classifier
clf.predict(X_test)

# Accuracy
clf.score(X_test, y_test)
```

```
1.0
```

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

h = 0.02  # Step size in the mesh

# Create color maps
cmap_light = ListedColormap(["orange", "cyan", "cornflowerblue"])
cmap_bold = ["darkorange", "c", "darkblue"]

for n_neighbors in [1, 5, 9]:
    # We create an instance of KNeighborsClassifier and fit the data.
    clf = KNeighborsClassifier(n_neighbors)
    clf.fit(X, y)

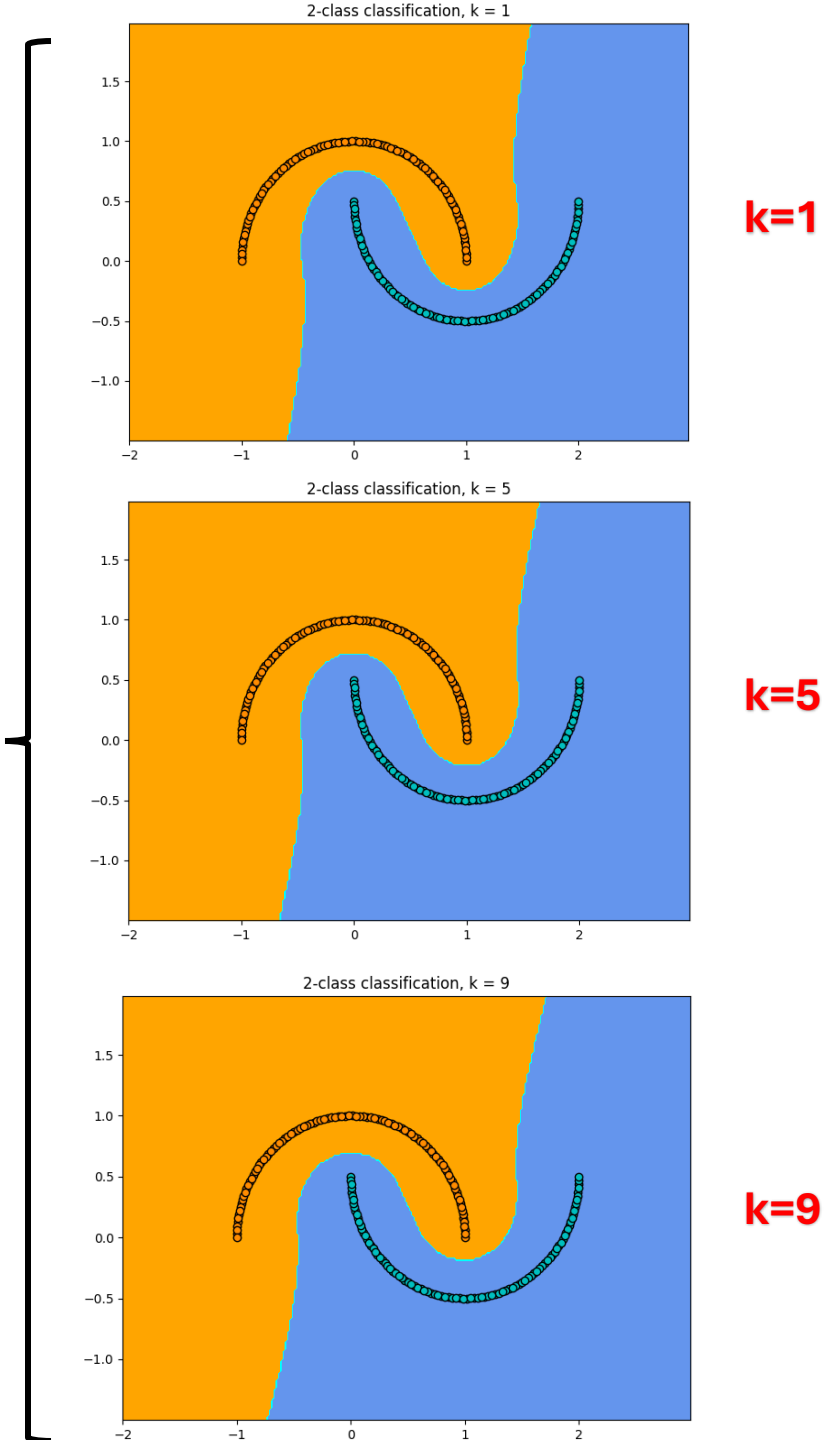
    # Plot the decision boundary. For that, we will assign a color to each
    # point in the mesh [x_min, x_max]x[y_min, y_max].
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])

    # Put the result into a color plot
    Z = Z.reshape(xx.shape)
    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, cmap=cmap_light)

    # Plot also the training points
    for i in range(2):
        plt.scatter(X[:, 0][y==i], X[:, 1][y==i], c=cmap_bold[i],
                    edgecolors="black", label=y[i])

    plt.xlim(xx.min(), xx.max())
    plt.ylim(yy.min(), yy.max())
    plt.title("2-class classification, k = %i" % (n_neighbors))
```



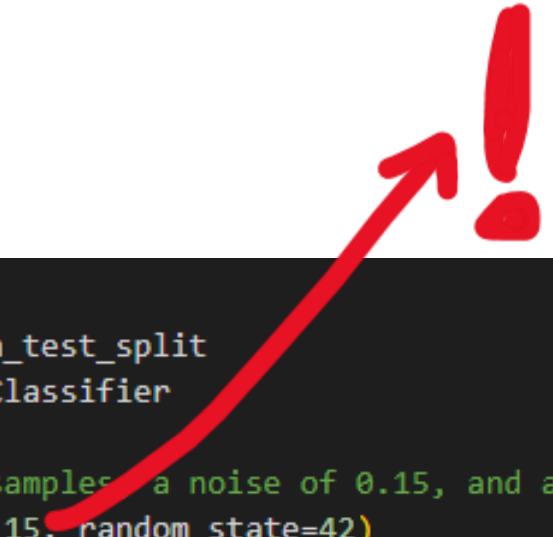


k=1

k=5

k=9

...or if we introduce a bit noise in data....



```python
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Generating the moons dataset with 200 samples, a noise of 0.15, and a random state for reproducibility
X, y = make_moons(n_samples=200, noise=0.15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.45 ,random_state=0)

clf = KNeighborsClassifier(n_neighbors=2)
clf.fit(X_train, y_train)

# Classifier
clf.predict(X_test)

# Accuracy
clf.score(X_test, y_test)
```

```
0.9888888888888889
```

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

h = 0.02  # Step size in the mesh

# Create color maps
cmap_light = ListedColormap(["orange", "cyan", "cornflowerblue"])
cmap_bold = ["darkorange", "c", "darkblue"]

for n_neighbors in [1, 5, 9]:
    # We create an instance of KNeighborsClassifier and fit the data.
    clf = KNeighborsClassifier(n_neighbors)
    clf.fit(X, y)

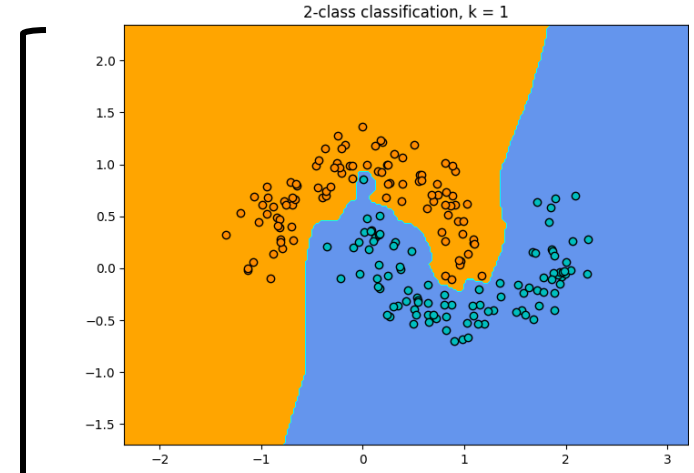
    # Plot the decision boundary. For that, we will assign a color to each
    # point in the mesh [x_min, x_max]x[y_min, y_max].
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])

    # Put the result into a color plot
    Z = Z.reshape(xx.shape)
    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, cmap=cmap_light)

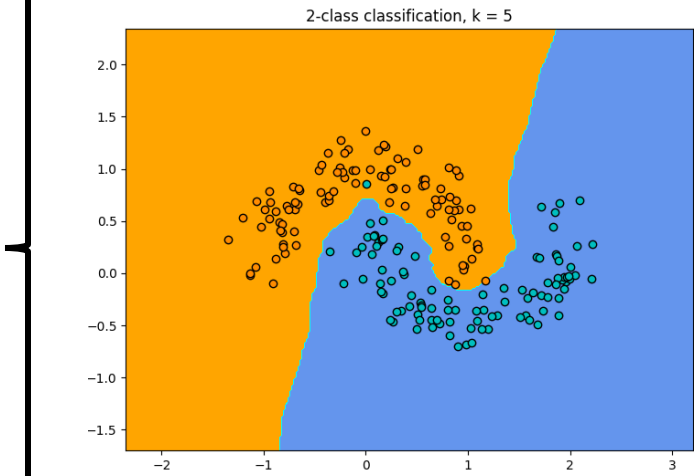
    # Plot also the training points
    for i in range(2):
        plt.scatter(X[:, 0][y==i], X[:, 1][y==i], c=cmap_bold[i],
                    edgecolors="black", label=y[i])

    plt.xlim(xx.min(), xx.max())
    plt.ylim(yy.min(), yy.max())
    plt.title("2-class classification, k = %i" % (n_neighbors))
```

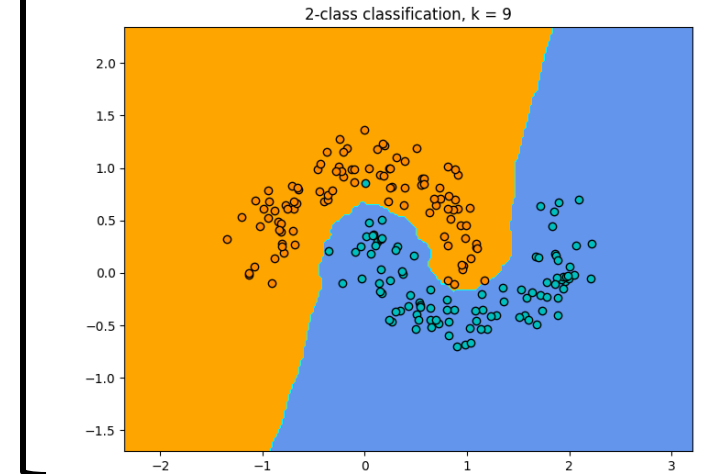




k=1



k=5



k=9

2. Use the following dataset:

```
X1, y1 = make_gaussian_quantiles(cov=7.,
                                 n_samples=250, n_features=2,
                                 n_classes=3, random_state=1),
```

in order to apply the nonlinear SVM classification. Use the polynomial kernel and set the `degree`, `coef0`, and `C` parameters to 3, 10, and 5 respectively. Plot the decision boundary of the nonlinear SVM classification.

```python
from sklearn.datasets import make_gaussian_quantiles
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler

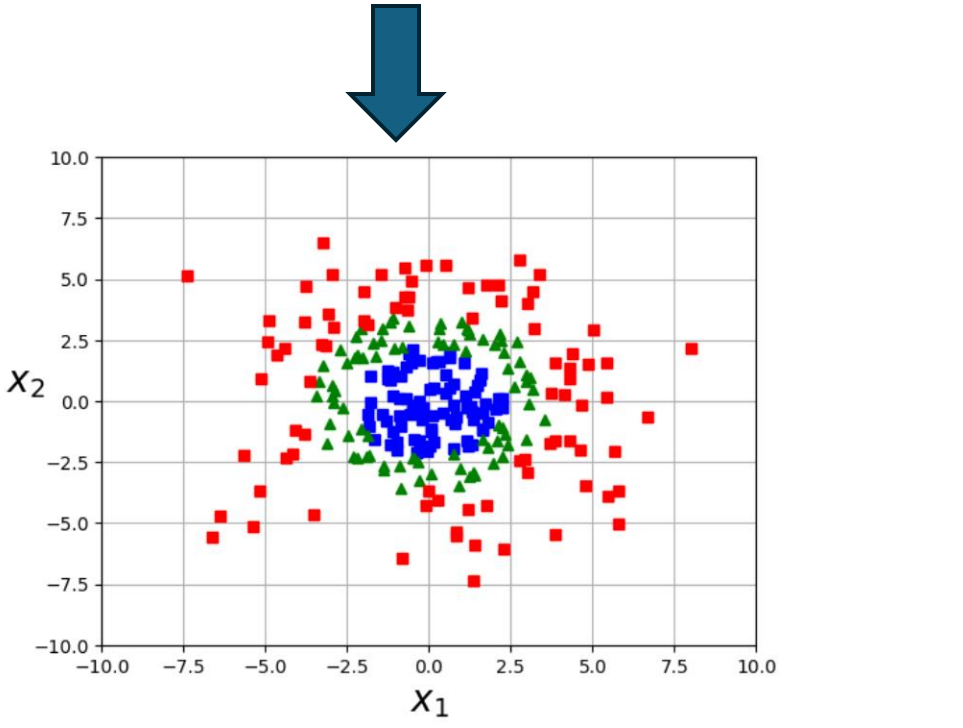
# Creating the synthetic dataset with 2 features, 3 classes, and 250 samples
X1 ,y1 = make_gaussian_quantiles(cov=7.,n_samples=250,n_features=2,n_classes=3,random_state=1)

# Creating a pipeline for SVM with a polynomial kernel.
# First, data is scaled using StandardScaler to standardize features (mean=0, variance=1)
# Then, the SVM is applied with a polynomial kernel using degree=3, coef0=10, and C=5
poly_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="poly", degree=3, coef0=10, C=5))
    ])

# Fit the model to the dataset
poly_kernel_svm_clf.fit(X1, y1)
```

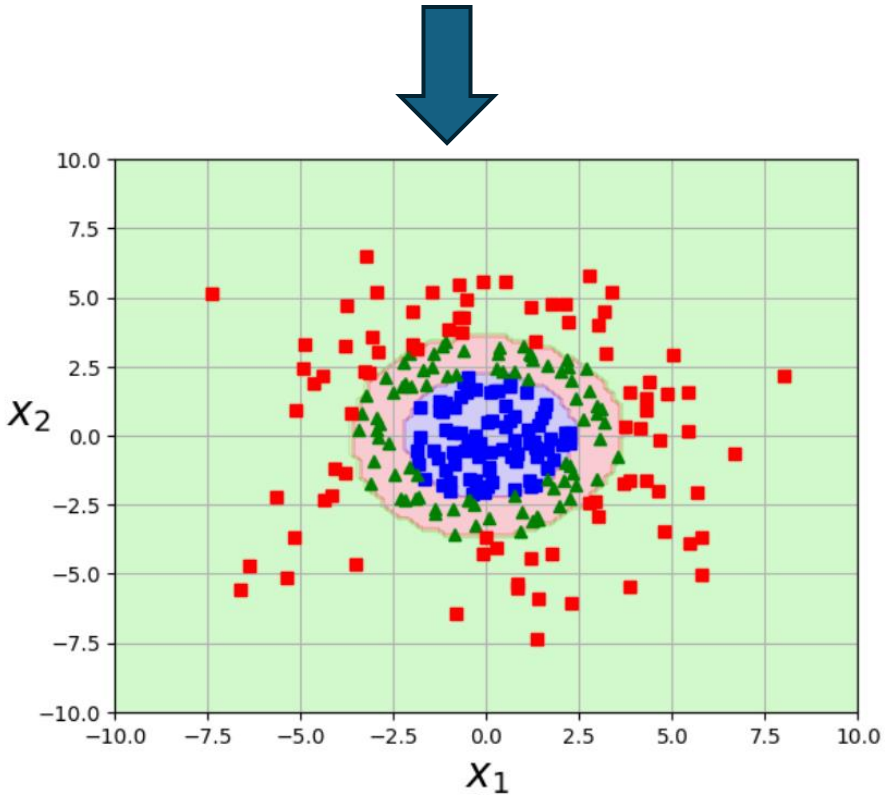
```python
# Function to plot the dataset with color-coded classes
def plot_dataset(X, y, axes):
    # Plot data points based on their class label
    plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs") # Class 0 in blue squares
    plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^") # Class 1 in green triangles
    plt.plot(X[:, 0][y==2], X[:, 1][y==2], "rs") # Class 2 in red squares
    plt.axis(axes) # Set the axis limits
    plt.grid(True, which='both')  # Add grid lines for better visualization
    plt.xlabel(r"$x_1$", fontsize=20)
    plt.ylabel(r"$x_2$", fontsize=20, rotation=0)

# Plot the dataset with axes range from -10 to 10 for both X and Y axes
plot_dataset(X1, y1, [-10,10,-10,10])
plt.show()
```



```python
# Function to plot decision boundaries of the trained classifier
def plot_predictions(clf, axes):
  # Create a mesh grid over the input feature space to plot decision regions
    x0s = np.linspace(axes[0], axes[1], 100) # Create 100 points for X1
    x1s = np.linspace(axes[2], axes[3], 100) # Create 100 points for X2
    x0, x1 = np.meshgrid(x0s, x1s) # Create a mesh grid using X1 and X2 points
    X = np.c_[x0.ravel(), x1.ravel()] # Combine into a 2D array for predictions
    y_pred = clf.predict(X).reshape(x0.shape) # Predict class labels for each point and reshape for plotting
    plt.contourf(x0, x1, y_pred, cmap=plt.cm.brg, alpha=0.2) # Plot the decision boundaries with contourf

plot_predictions(poly_kernel_svm_clf, [-10,10,-10,10])
plot_dataset(X1, y1, [-10,10,-10,10])
plt.show()
```



3. Apply linear SVM regression to the following dataset:

```
X = 6 * np.random.rand(n, 1)
y = (1 - 2 * X + np.random.randn(n, 1)).ravel().
```

Set the width of the margin to 0.8 and plot the fit line.

```python
from sklearn.svm import LinearSVR

# Setting a seed for reproducibility of random numbers
np.random.seed(42)
# Defining the number of samples
n = 50

# Generating a random dataset X
X = 6 * np.random.rand(n, 1)
# Creating the target variable y using a linear relationship with X, adding some noise
y = (1 - 2 * X + np.random.randn(n, 1)).ravel() # Flattening the array for compatibility

# Creating an instance of LinearSVR with epsilon=0.8
# Epsilon controls the margin width for error tolerance in the regression task
svm_reg = LinearSVR(epsilon=0.8, random_state=42)
svm_reg.fit(X, y)  # Fitting the SVM regression model to the data

# Function to find the support vectors, i.e., data points that lie outside the epsilon margin
def find_support_vectors(svm_reg, X, y):
    y_pred = svm_reg.predict(X) # Predict the target values for the dataset X
    off_margin = (np.abs(y - y_pred) >= svm_reg.epsilon) # Check if predictions are outside the margin
    return np.argwhere(off_margin) # Return the indices of support vectors

# Finding and storing the support vectors
svm_reg.support_ = find_support_vectors(svm_reg, X, y)
```

```
# Function to plot the SVM regression line, margin, and support vectors
def plot_svm_regression(svm_reg, X, y, axes):
    # Create 100 points in the range for plotting the line
    x1s = np.linspace(axes[0], axes[1], 100).reshape(100, 1)
    # Predict y values for the new points (for plotting the regression line)
    y_pred = svm_reg.predict(x1s)

    # Plot the regression line
    plt.plot(x1s, y_pred, "k-", linewidth=2, label=r"$\hat{y}$") # Plot the predicted regression line in black

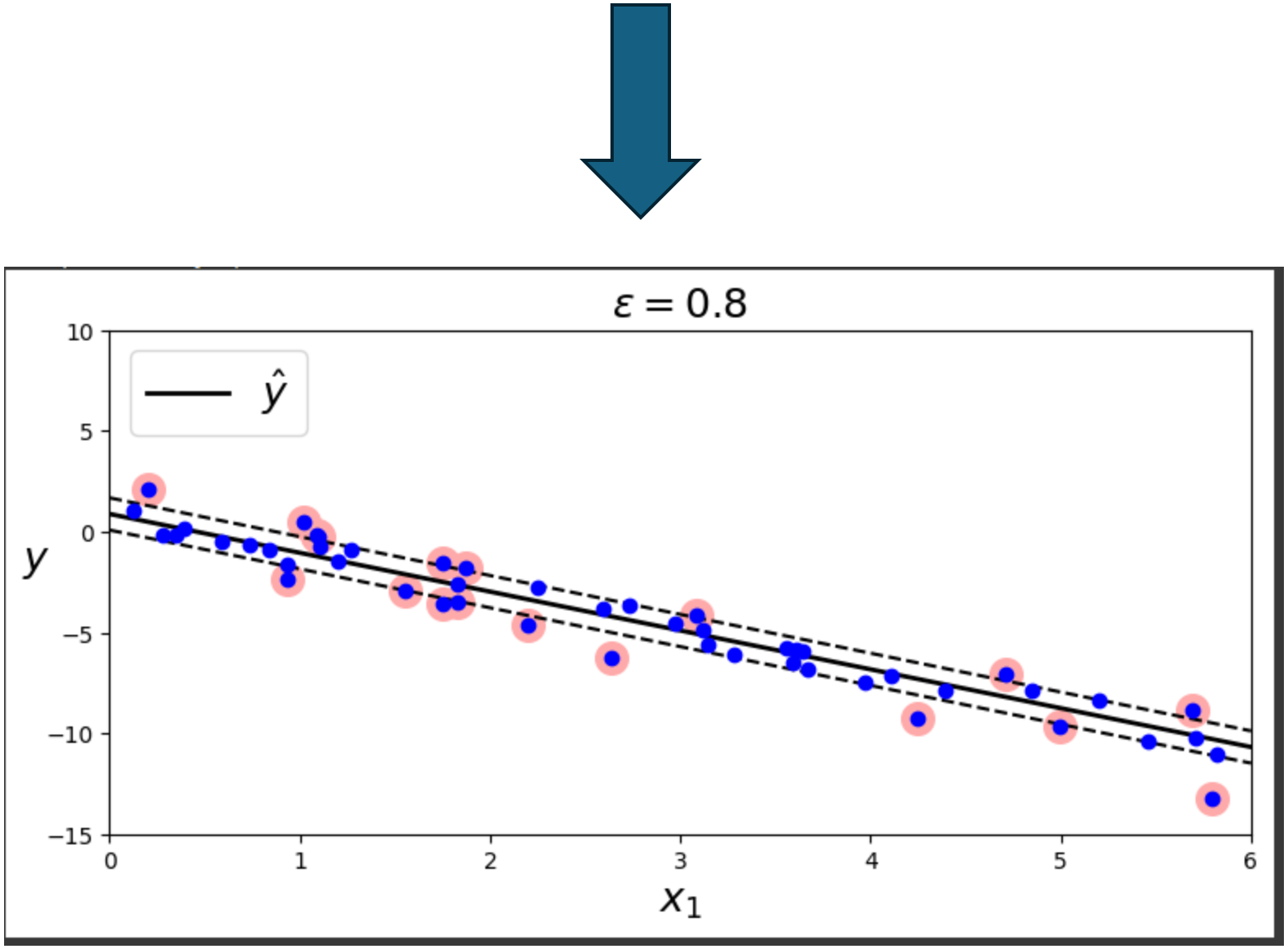
    # Plot the epsilon margin boundaries (above and below the regression line)
    plt.plot(x1s, y_pred + svm_reg.epsilon, "k--") # Upper boundary of the margin
    plt.plot(x1s, y_pred - svm_reg.epsilon, "k--") # Lower boundary of the margin

    # Highlight the support vectors
    plt.scatter(X[svm_reg.support_], y[svm_reg.support_], s=200, facecolors='#FFAAAA') # Support vectors in red

    # Plot the actual data points
    plt.plot(X, y, "bo") # Actual data points in blue

    # Label the axes
    plt.xlabel(r"$x_1$", fontsize=18)
    plt.legend(loc="upper left", fontsize=18)
    plt.axis(axes)

# Creating a figure and plotting the SVM regression with a margin of 0.8
fig, axes = plt.subplots(ncols=1, figsize=(9, 4)) # Create a subplot
plt.sca(axes) # Set the current axis
plot_svm_regression(svm_reg, X, y, [0, 6, -15, 10]) # Plot the SVM regression with specified axis limits
plt.title(r"$\epsilon = {}$".format(svm_reg.epsilon), fontsize=18)
plt.ylabel(r"$y$", fontsize=18, rotation=0)
```


$\varepsilon = 0.8$
$\hat{y}$
10
5
0
−5
−10
−15
y
0
1
2
3
4
5
6
x_1

Thank you !